Informatik
Hauptcampus

HOCHSCHULE TRIER

Teamprojekt „Entwicklung einer mobilen Anwendung zur automatisierten Erkennung und Übersetzung japanischer Texte in Bildern”

Leonie Kaschek
Nagina Nasir
Franziska Kaschek

Bachelor-Teamprojekt

Betreuer: Prof. Dr. Claudia Schon

Trier, 01.09.2025

Kurzfassung

Diese Arbeit entstand im Rahmen eines Bachelor-Teamprojekts der Hochschule Trier und befasst sich mit der Entwicklung einer mobilen App, die japanische Texte in Bildern erkennt, mit der Lesehilfe Furigana versieht und ins Englische übersetzt. Die App richtet sich an Japanischlernende und Touristen, die im Alltag Unterstützung beim Lesen japanischer Texte benötigen. Die technische Umsetzung basiert auf React Native und Expo und ist für Android-Endgeräte ausgelegt.
Für die optische Zeichenerkennung (OCR) wird Google ML Kit eingesetzt. Die Furigana-Generierung erfolgt mithilfe der Kuromoji-Bibliothek, die eine morphologische Analyse durchführt. Die Übersetzung ins Englische übernimmt ein Modell aus der Helsinki-NLP-Reihe von Hugging Face, das auf einem eigenen Server betrieben wird
Nach der Implementierung erfolgte eine umfassende Leistungsbewertung der App. Die OCR-Genauigkeit wurde anhand der Character Error Rate (CER) bewertet, die Furigana-Generierung einer detaillierten Fehleranalyse unterzogen und die Übersetzungsqualität mit den Metriken BLEU und COMET beurteilt. Ergänzend lieferten Usability-Tests mit realitätsnahen Anwendungsszenarien qualitative Rückmeldungen zur Gebrauchstauglichkeit der Anwendung.
Die App zeigt ein hohes Potenzial als sprachliches Hilfsmittel. Zugleich weist sie noch deutliche Schwächen auf, insbesondere in der Qualität der Ergebnisse der sprachtechnologischen Komponenten. Diese liefern wertvolle Hinweise für künftige Optimierungen.

Inhaltsverzeichnis

1 Einleitung 1
1.1 Zielsetzung und Motivation 1
1.2 Zielgruppe und Anwendungsfälle 1

2 Einführung in das japanische Schriftsystem 2
2.1 Kanji 2
2.2 Kana 3
2.3 Furigana 4
2.4 Schreibrichtungen im Japanischen 4

3 Anforderungsanalyse 6
3.1 Stakeholder-Analyse 6
3.2 Benutzungsszenarien (Use Cases) 8
3.3 Methodisches Vorgehen 10
3.4 Funktionale Anforderungen 11
3.5 Nicht-funktionale Anforderungen 12
3.6 Prototyping 14

4 Technische Konzeption und Implementierung 16
4.1 Frameworks 16
4.1.1 React Native 16
4.1.2 Expo 17
4.2 Verzeichnisstruktur 17
4.3 Funktionale Komponenten 20
4.3.1 Bildimport-Komponenten (Kamera und Galerie) 21
4.3.2 Komponente zur Texterkennung (OCR) 22
4.3.3 Komponente zur Furigana-Erzeugung 27
4.3.4 Übersetzungskomponente 33
4.3.5 Speicherkomponente 36
4.3.6 Komponente zum PDF-Export 38

5 Qualitätssicherung 41
5.1 Evaluation der Hauptfunktionen 41

5.1.1 Evaluation der OCR-Funktion 42
5.1.2 Evaluation der Furigana-Funktion 45
5.1.3 Evaluation der Übersetzungsfunktion 47
5.2 Usability Testing 54
5.2.1 Testergebnisse 56

6 Ergebnisse 57
6.1 Farbpalette 57
6.2 Abschließende App-Version 57
6.3 Technische Einschränkungen und offene Aufgaben 60

7 Zusammenfassung 61

Literaturverzeichnis 62

Eigenständigkeitserklärung 64

1

Einleitung

Im Rahmen des Pflichtmoduls „Teamprojekt“ des Fachbereichs Informatik der Hochschule Trier entwickelten die Studentinnen Leonie Kaschek, Nagina Nasir und Franziska Kaschek mithilfe moderner Technologien eine mobile Anwendung im Bereich interkulturelle Kommunikation. Die Projektidee entstand in einem Gespräch mit einem taiwanesischen Austauschstudenten der Hochschule, der zum Übersetzen komplexer Schreiben eine OCR-basierte App nutzte.

1.1 Zielsetzung und Motivation

Ziel des Teamprojekts war die Entwicklung einer mobilen Anwendung, die japanische Texte in Bildern erkennt, automatisch übersetzt und mit der Lesehilfe Furigana ergänzt. Die Anwendung sollte im Idealfall sowohl für iOS- als auch für Android-Geräte verfügbar gemacht werden.

Das japanische Schriftsystem kann aufgrund seiner Komplexität das Textverständnis erheblich erschweren. Die Anwendung beabsichtigt, Barrieren beim Lesen japanischer Texte abzubauen und die Sprache insbesondere für Nicht-Muttersprachler zugänglicher zu machen.

1.2 Zielgruppe und Anwendungsfälle

Die App richtet sich vor allem an Japanisch-Lernende und Touristen in Japan. Darüber hinaus können auch Muttersprachler, die Schwierigkeiten mit dem Lesen von Kanji haben, von ihr profitieren.

Typische Anwendungsfälle sind das Entziffern von Speisekarten, Beschilderungen, Produktverpackungen und anderen alltagsrelevanten Texten in Bildern. Die Anwendung unterstützt Nutzerinnen und Nutzer dabei, Inhalte schnell zu erfassen und erleichtert so den Zugang zur japanischen Sprache im Alltag.

2

Einführung in das japanische Schriftsystem

Um die Funktionsweise der entwickelten Anwendung nachvollziehen zu können, ist ein grundlegendes Verständnis des japanischen Schriftsystems erforderlich.

Das japanische Schriftsystem vereint fünf verschiedene Zeichensysteme: Kanji, Hiragana, Katakana, lateinische Buchstaben (Rōmaji) und arabische Ziffern [Ros17, S.14]. Abbildung 2.1 zeigt einen Textausschnitt, in dem diese Zeichensysteme gemeinsam verwendet werden. Katakana ist hier zur Veranschaulichung durch eine gerade Unterstreichung gekennzeichnet, Hiragana durch eine wellenförmige Linie hervorgehoben. Rōmaji sowie arabische Ziffern sind kursiv dargestellt. Kanji bleiben unmarkiert.[1]

トリーア（ドイツ語: *Trier*, フランス語: *Trèves*）は、ドイツ連邦共和国ラインラント＝プファルツ州の都市。人口は約 *10* 万人（*2005* 年）。トリアーとも表記される。

かつては舞台ドイツ語式発音に基づいてトリール、トリエルと呼ばれたが、近年では現在の標準ドイツ語の発音に近い「トリーア」が主に用いられる。

Abbildung 2.1: Beispieltext, in dem die unterschiedlichen Schriftsysteme kombiniert vorkommen.

2.1 Kanji

Die Schriftzeichen *Kanji* stammen ursprünglich aus dem Chinesischen und stellen bedeutungstragende Einheiten (Morpheme) dar. Das Erlernen und Erinnern von Kanji ist nicht nur für Japanischlernende, sondern auch für Muttersprachler eine lebenslange Aufgabe, denn es existieren mehrere zehntausend Kanji-Zeichen. So umfasst das größte Kanji-Lexikon Japans etwa 50.000 Zeichen [Zis20, S.41]. Davon erlernen japanische Schüler während ihrer Schullaufbahn 2.136 Zeichen. Die Strichanzahl alltäglicher Kanji reicht von einem bis zu knapp dreißig Strichen

[1] O. V. (o. D.). *Trier*. Wikipedia. Abgerufen am 26. Mai 2025, von `https://ja.wikipedia.org/wiki/`

[Pax19, S.33], die nach einer vorgeschriebenen Reihenfolge und Strichrichtung geschrieben werden [Pax19, S.35]. Zudem haben Kanji zwei verschiedene Lesungen: eine sinojapanische On-Lesung und eine japanische Kun-Lesung. Welche Lesung verwendet wird, hängt vom Worttyp und der Kombination mit anderen Kanji ab [Pax19, S.33]. Abbildung 2.2 zeigt das Kanji-Zeichen für Frühling mit seinen Lesungen, der Strichanzahl sowie der Strichfolge.[2] [3]



Bedeutung	Frühling
On-Lesung	SHUN
Kun-Lesung	haru
Striche	9
Beispielkompositum	春分 = shunbun = Frühlingsanfang

Abbildung 2.2: Kanji für Frühling mit seinen Lesungen, Strichanzahl und Strichfolge.

2.2 Kana

Hiragana und Katakana, zusammenfassend als *Kana* bezeichnet, bestehen jeweils aus nur 46 Grundzeichen. Sie entwickelten sich historisch durch Vereinfachung aus Kanji, wie in Abbildung2.3 und 2.4 veranschaulicht. Dabei verloren sie ihre ursprüngliche semantische Bedeutung und dienen heute ausschließlich der lautlichen Wiedergabe von Silben im Japanischen [Bee06, S.2–4]. Jedes Zeichen steht für einen Silbenlaut, wobei Hiragana und Katakana dieselben Silbenlaute in unterschiedlicher Schreibweise darstellen.



Abbildung 2.3: Hiragana entstand durch die Vereinfachung von Kanji, indem deren Bestandteile in fließender, kursiver Form verbunden wurden [LA24, S.100].

[2] O. V. (o. D.). 春. Kanshudo. Abgerufen am 27. Mai 2025, von `https://www.kanshudo.com/kanji/`
[3] Bibiko, H. (o. D.). 春. Japanisch-Deutsches Kanji-Lexikon. Abgerufen am 27. Mai 2025, von `https://lingweb.eva.mpg.de/kanji/index.html?kanji=`

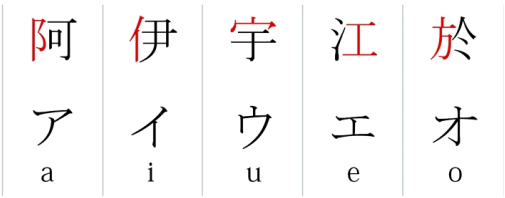


Abbildung 2.4: Katakana entstand durch die Vereinfachung von Kanji, wobei einzelne Bestandteile herausgelöst wurden [LA24, S.101].

2.3 Furigana

Texte werden oft mit *Furigana* ergänzt, da Leser nicht immer die korrekte Lesung bestimmter Kanji kennen. Dabei handelt es sich meist um kleine Hiragana-Zeichen, die über dem Kanji (im vertikalen Text neben dem Zeichen) stehen und dessen Aussprache angeben, siehe Abbildung 2.5. Um in der japanischen Sprache als funktional alphabetisiert zu gelten, müssen etwa 2000 Kanji beherrscht werden [Ros17, S.8]. Gerade für Nicht-Muttersprachler machen Furigana japanische Texte dadurch deutlich zugänglicher.

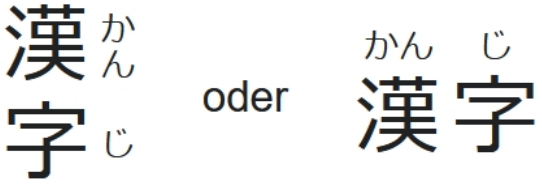


Abbildung 2.5: Darstellung des Wortes Kanji mit Furigana in vertikaler und horizontaler Anordnung.

2.4 Schreibrichtungen im Japanischen

Traditionell wird Japanisch vertikal (*tategaki*) von oben nach unten und Spalte für Spalte von rechts nach links geschrieben. Heutzutage werden Texte jedoch zunehmend horizontal (*yokogaki*) von links nach rechts und Zeile für Zeile von oben nach unten verfasst. Abbildung 2.6 verdeutlicht die Unterschiede zwischen tategaki und yokogaki. [4] Welche Schreibrichtung zum Einsatz kommt, hängt vom Medium und Thema ab: Vertikal schreibt man meist Romane, Gedichte, Zeitungen, Comics und Wörterbücher, während E-Mails, Anleitungen und wissenschaftliche Texte überwiegend horizontal verfasst werden.[5]

[4] Kawanami, S. (2013, 9. Mai). *Escrita japonesa –Tategaki e Yokogaki.* Japao em foco. Abgerufen am 27. Mai 2025, von `https://www.japaoemfoco.com/escrita-japonesa-tategaki-e-yokogaki/`

[5] O. V. (o. D.). *Japanese/Japanese writing system.* Wikibooks. Abgerufen am 27. Mai 2025, von `https://en.wikibooks.org/wiki/Japanese/Japanese_writing_system`

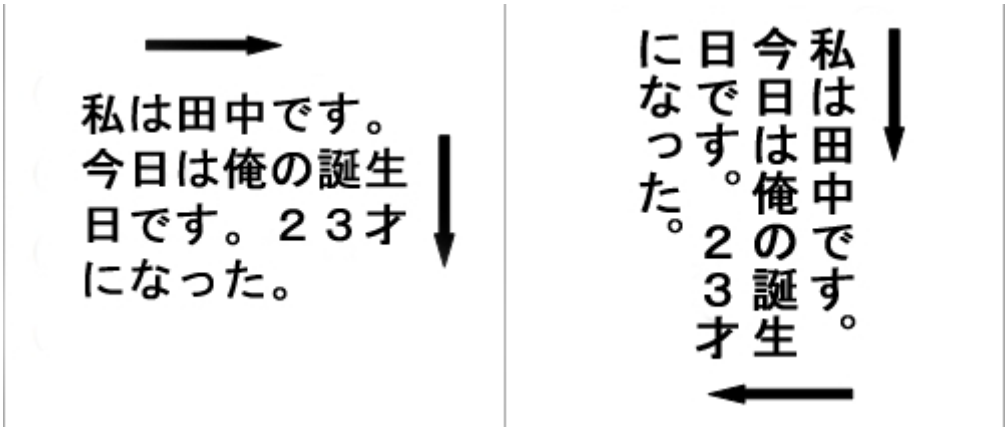


Abbildung 2.6: Links: die horizontale Schreibweise yokogaki. Rechts: die vertikale Schreibweise tategaki.

3

Anforderungsanalyse

Der Erfolg eines Softwareprojekts hängt maßgeblich von einer gründlichen *Anforderungsanalyse* ab. Hussain et al. zeigen in ihrem Fachartikel, dass unvollständige Anforderungen zu den häufigsten Ursachen für das Scheitern von Projekten zählen [HMK16, S. 308]. Daher ist es unerlässlich, vor dem Beginn von Design und Implementierung eine fundierte Anforderungsanalyse durchzuführen.

Die IEEE definiert in ihrem Standard IEEE Std. 610.12-1990 eine *Anforderung* als eine Bedingung oder Fähigkeit, die von einem Benutzer benötigt wird, um ein Ziel zu erreichen oder ein Problem zu lösen. Alternativ kann es sich um eine Eigenschaft handeln, die ein System oder eine Systemkomponente erfüllen muss, um vertragliche Vereinbarungen, anerkannte Standards oder technische Spezifikationen einzuhalten. Der Standard schließt zudem auch die formalisierte, dokumentierte Darstellung solcher Anforderungen mit ein [IEE90, S.62].

In demselben IEEE Standard wird auch der Begriff der Anforderungsanalyse beschrieben. Dabei handelt es sich um einen zweistufigen Prozess: Zunächst werden die Bedürfnisse der Benutzer untersucht, um daraus eine Definition der System-, Hardware- oder Softwareanforderungen abzuleiten. Anschließend erfolgt eine Analyse und Verfeinerung dieser Anforderungen, um sie weiter zu präzisieren und zu konkretisieren [IEE90, S.62].

Nach der Identifikation relevanter Stakeholder werden die Bedürfnisse der Endnutzer analysiert und zentrale Anwendungsfälle beschrieben. Darauf aufbauend werden funktionale und nicht-funktionale Anforderungen formuliert.

3.1 Stakeholder-Analyse

> „[Ein Stakeholder ist] eine Person oder Organisation, die die Anforderungen eines Systems beeinflusst oder von diesem System betroffen ist."[1]

[1] O. V. (o. D.). *Das CPRE Glossar –Abschnitt „S "*. IREB CPRE. Abgerufen am 20. Juli 2025, von `https://cpre.ireb.org/de/downloads-and-resources/glossary?first-letter=S`

In Softwareprojekten zählen typischerweise Nutzer, Entwickler, Sponsoren, Manager, Auftraggeber sowie Behörden zu den Stakeholdern [PR21, S.118]. Da es sich bei diesem Projekt um eine studentische Einzelarbeit ohne realen externen Auftraggeber handelt, wurde eine vereinfachte Stakeholderanalyse durchgeführt. Dabei wurden die wichtigsten Beteiligten identifiziert sowie deren Einfluss und Interessen bewertet (siehe Abbildung 3.1).

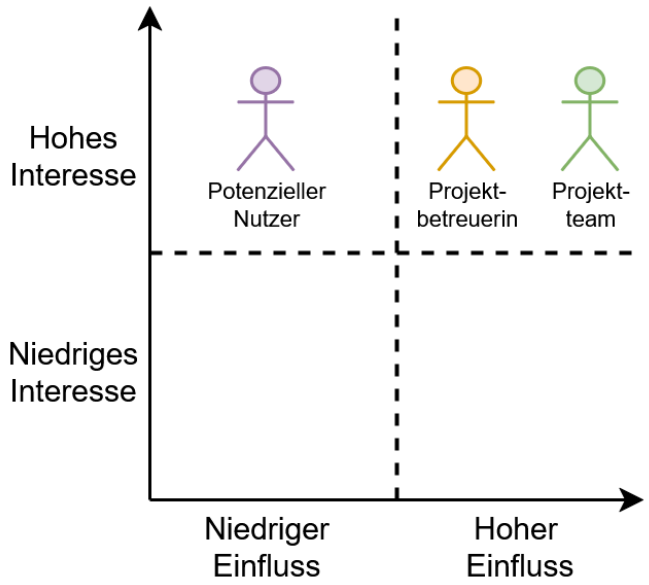


Abbildung 3.1: Stakeholder-Matrix zur Einordnung der relevanten Projektbeteiligten nach Einfluss und Interesse.

Potenzielle Nutzer

Die potenziellen Endnutzer der App sind einerseits Personen, die Japanisch im Selbststudium oder während ihres Hochschulstudiums lernen und Unterstützung beim Lesen und Verstehen japanischer Texte benötigen. Andererseits zählen auch Touristen dazu, die während eines Aufenthalts in Japan sprachliche Hilfen benötigen. Diese Zielgruppen unterscheiden sich in ihren Sprachkenntnissen und Nutzungsszenarien, teilen jedoch das grundlegende Bedürfnis, unbekannte Zeichen und Ausdrücke schnell zu erschließen. Ihr Interesse an einer funktionalen und benutzerfreundlichen Anwendung ist hoch. Ein direkter Einfluss auf den Entwicklungsprozess bestand im Rahmen dieses Projekts nicht, da ihre Einbindung erst am Ende im Usability-Test erfolgte. Indirekt beeinflussten sie die Entwicklung jedoch insofern, als ihre Bedürfnisse und Nutzungskontexte die Konzeption und Gestaltung der App wesentlich prägten.

Projektbetreuerin

Die Projektbetreuerin begleitete das Vorhaben aus wissenschaftlicher Perspektive, bewertete den Fortschritt und stellte die Einhaltung fachlicher sowie formaler

Anforderungen sicher. Ihr Einfluss auf die Projektgestaltung und Bewertung war hoch, wobei ihr Interesse insbesondere der methodischen und inhaltlichen Qualität der Umsetzung galt.

Projektteam

Das Projektteam bestand aus drei Studierenden, die gemeinsam für Planung, Umsetzung, Evaluation und Dokumentation der App verantwortlich waren. Als zentrale Akteure trugen sie die inhaltliche, technische und gestalterische Verantwortung und hatten einen hohen Einfluss auf die Projektentscheidungen und den Projektfortschritt.

3.2 Benutzungsszenarien (Use Cases)

> „[Use Cases sind] eine Menge möglicher Interaktionen zwischen externen Akteuren und einem System, die den beteiligten Akteuren einen Nutzen bringen."[2]

In der Praxis gilt nicht jede Interaktion automatisch als Use Case. Nur Handlungen, die für den Akteur einen konkreten Mehrwert erzeugen, werden als solche betrachtet. Ein Use Case beginnt mit einem klar definierten Auslöser (Trigger) und führt zu einer Veränderung des Systemzustands oder eines Objekts [Her22, S.114]. Use Cases beschreiben ausschließlich das von außen sichtbare Verhalten des Systems aus Sicht der Nutzer. Sie dienen der Anforderungserhebung, enthalten jedoch keine technischen Umsetzungsdetails [WB13, S.161]. Tabelle 3.1 listet die acht identifizierten Use Cases der App auf. Diese bilden die wesentlichen Interaktionen der Nutzer mit dem System ab. Auf ihrer Basis erfolgt anschließend die Ableitung funktionaler und nicht-funktionaler Anforderungen.

[2] O. V. (o. D.). *Das CPRE Glossar –Abschnitt „U "*. IREB CPRE. Abgerufen am 20. Juli 2025, von `https://cpre.ireb.org/de/downloads-and-resources/glossary?first-letter=U`

Tabelle 3.1: Tabellarische Auflistung der Use Cases mit jeweiliger ID, Titel und einer standardisierten Beschreibung bestehend aus Akteur, Ziel, Auslöser, Vorbedingung, Ablauf und Nachbedingung.

ID	**Titel**	**Beschreibung**
UC01	Foto aufnehmen	**Akteur:** Nutzer **Ziel:** Neues Bild für Texterkennung aufnehmen **Auslöser:** Nutzer wählt „Foto aufnehmen“ **Vorbedingung:** Kamerazugriff erlaubt **Ablauf:** Kamera öffnet sich, Nutzer macht ein Foto und kann es anschließend optimieren **Nachbedingung:** Bild ist temporär gespeichert und kann weiterverarbeitet werden
UC02	Bild aus Galerie wählen	**Akteur:** Nutzer **Ziel:** Bild zur Texterkennung laden **Auslöser:** Nutzer wählt „Bild aus Galerie laden“ **Vorbedingung:** Galeriezugriff erlaubt **Ablauf:** Galerie öffnet sich, Nutzer wählt ein Bild und kann es anschließend optimieren **Nachbedingung:** Bild ist geladen und kann weiterverarbeitet werden
UC03	Texterkennung auslösen	**Akteur:** Nutzer **Ziel:** Japanischen Text erkennen und als maschinenlesbaren Text temporär speichern **Auslöser:** Nutzer wählt „Scan“ **Vorbedingung:** Bild liegt vor **Ablauf:** System analysiert Bild, extrahiert Text, speichert temporär **Nachbedingung:** Text und Bild werden angezeigt; Bild kann vergrößert werden
UC04	Furigana hinzufügen	**Akteur:** Nutzer **Ziel:** Kanji mit Furigana versehen **Auslöser:** Nutzer wählt „Furigana anzeigen“ **Vorbedingung:** Text erfolgreich erkannt **Ablauf:** System ermittelt Lesung der Kanji und ergänzt diese als Furigana **Nachbedingung:** Furigana sind sichtbar

Fortsetzung auf nächster Seite

ID	Titel	Beschreibung
UC05	Text übersetzen	**Akteur:** Nutzer **Ziel:** Japanischen Text automatisch ins Englische übersetzen **Auslöser:** Nutzer wählt „Scan“ und Text wurde erfolgreich erkannt **Vorbedingung:** Erkannter Text liegt vor **Ablauf:** System startet Übersetzung automatisch **Nachbedingung:** Übersetzung wird angezeigt
UC06	Text bearbeiten	**Akteur:** Nutzer **Ziel:** Erkannten Text manuell korrigieren **Auslöser:** Nutzer wählt „Bearbeiten“ **Vorbedingung:** Texterkennung abgeschlossen **Ablauf:** Nutzer korrigiert falsch erkannte Zeichen oder fügt nicht erkannte Zeichen ein und speichert die Veränderungen **Nachbedingung:** Modifizierter Text liegt vor
UC07	Ergebnisse speichern	**Akteur:** Nutzer **Ziel:** Texte mit Bild, Übersetzung und Furigana in der Historie speichern **Auslöser:** Nutzer wählt „Speichern“ **Vorbedingung:** Texterkennung abgeschlossen **Ablauf:** System speichert Ergebnisse in der App-Historie **Nachbedingung:** Daten sind in der Historie verfügbar
UC08	Ergebnisse exportieren	**Akteur:** Nutzer **Ziel:** Beiträge aus der Historie lokal exportieren **Auslöser:** Nutzer wählt „Exportieren“ **Vorbedingung:** Mindestens ein Beitrag gespeichert **Ablauf:** Beiträge auswählen, Export auslösen, Speicherort festlegen. **Nachbedingung:** Beiträge sind lokal gespeichert

3.3 Methodisches Vorgehen

Zur Erhebung und Strukturierung der Anforderungen wurde folgendes methodisches Vorgehen gewählt:

- **Anforderungsarten:** Die Anforderungen wurden in funktionale und nichtfunktionale Anforderungen unterteilt.

- **Qualitätskriterien für Anforderungen:** Bei der Formulierung der Anforderungen wurden etablierte Qualitätskriterien berücksichtigt. Dazu gehören Adäquatheit, Verständlichkeit, Notwendigkeit, Eindeutigkeit, Vollständigkeit sowie Prüfbarkeit [PR21, S.114].

- **Anforderungsmanagement:** Zur Erfassung, Verfolgung und Verwaltung der Anforderungen wurde GitLab verwendet. Jede Anforderung wurde als Issue angelegt und im Projektverlauf aktualisiert.

- **Priorisierung:** Die MoSCoW-Priorisierung wurde verwendet, um die Relevanz einzelner Anforderungen zu gewichten (Must-have, Should-have, Could-have, Won't-have-now) [WB13, S.320-321].

- **Klassifikation nicht-funktionaler Anforderungen:** Für die strukturierte Einordnung der nicht-funktionalen Anforderungen wurde das Qualitätsmodell ISO/IEC 25010 zugrunde gelegt. Dieses definiert acht zentrale Qualitätsmerkmale: Functional Suitability (Funktionalität), Performance Efficiency (Effizienz), Compatibility (Kompatibilität), Usability (Benutzerfreundlichkeit), Reliability (Zuverlässigkeit), Security (Sicherheit), Maintainability (Wartbarkeit) und Portability (Übertragbarkeit) [Her22, S.41-43].

3.4 Funktionale Anforderungen

> „[Funktionale Anforderungen] sind Aussagen darüber, welche Dienste das System bereitstellen soll, wie es auf bestimmte Eingaben reagieren und wie es sich in bestimmten Situationen verhalten soll."[Som11, S.85]

Funktionale Anforderungen beschreiben also, was ein System tun soll. Die nachfolgende Tabelle 3.2 gibt einen Überblick über alle funktionalen Anforderungen der App und bildet damit die Grundlage für die spätere Implementierung.

Tabelle 3.2: Funktionale Anforderungen mit Angabe von ID, Anforderungsbeschreibung, Prioritätseinstufung und Herleitung aus den jeweiligen Use Cases.

ID	**Formulierung der Anforderung**	**Priorität**	**Abgeleitet aus**
FA01	Das System MUSS dem Nutzer die Möglichkeit bieten, ein Foto mit der Kamera aufzunehmen.	Must-have	UC01
FA02	Das System MUSS dem Nutzer die Möglichkeit bieten, ein Bild aus der Galerie auszuwählen.	Must-have	UC02
FA03	Nach der Bildaufnahme oder -auswahl SOLLTE das System dem Nutzer die Möglichkeit bieten, das Bild zuzuschneiden oder zu rotieren.	Should-have	UC01, UC02
FA04	Wenn ein Bild aufgenommen oder geladen wurde, KANN das System dem Nutzer die Möglichkeit bieten, einfache Bildoptimierungen wie Schärfung und Kontrasterhöhung durchzuführen.	Could-have	UC01, UC02

Fortsetzung auf nächster Seite

ID	Formulierung der Anforderung	Priorität	Abgeleitet aus
FA05	Das System MUSS bei einem vorliegenden Bild handschriftlichen und gedruckten Text automatisch per OCR erfassen, unabhängig von der Ausrichtung (horizontal oder vertikal) und einschließlich lateinischer Schriftzeichen.	Must-have	UC03
FA06	Das System KANN eine Intelligent Character Recognition (ICR) einsetzen, um die Genauigkeit der Texterkennung zu verbessern.	Could-have	UC03
FA07	Wenn Text erfolgreich erkannt wurde, MUSS das System in der Lage sein, Furigana oberhalb der erkannten Kanji anzuzeigen.	Must-have	UC04
FA08	Wenn Text vorliegt, MUSS das System in der Lage sein, den erkannten Text automatisch ins Englische zu übersetzen.	Must-have	UC05
FA09	Nach erfolgter Texterkennung SOLLTE das System dem Nutzer die Möglichkeit bieten, den Text manuell zu bearbeiten.	Should-have	UC06
FA10	Nach der Texterkennung SOLLTE das System dem Nutzer die Möglichkeit bieten, die Ergebnisse (Bild, Text, Furigana, Übersetzung) in einer Historie zu speichern.	Should-have	UC07
FA11	Wenn Beiträge gespeichert wurden, KANN das System dem Nutzer die Möglichkeit bieten, diese als PDF-Datei lokal zu exportieren.	Could-have	UC08

3.5 Nicht-funktionale Anforderungen

> „Nicht-funktionale Anforderungen, wie Leistung, Sicherheit oder Verfügbarkeit, legen in der Regel die Eigenschaften des Gesamtsystems fest oder schränken diese ein. Nicht-funktionale Anforderungen sind oft wichtiger als einzelne funktionale Anforderungen."[Som11, S.85]

Diese Anforderungen legen wesentliche Qualitätsmerkmale des Systems fest, etwa in Bezug auf Geschwindigkeit, Zuverlässigkeit, Benutzerfreundlichkeit oder Sicherheit, und grenzen dessen Verhalten ein. Zur besseren Strukturierung wurden sie anhand der Qualitätsmerkmale der Norm ISO/IEC 25010 klassifiziert. Nicht alle Merkmale sind für das Projekt relevant, weshalb nur ein Teil berücksichtigt wurde. Die folgende Tabelle zeigt die Anforderungen inklusive der zugehörigen Qualitätsmerkmale.

Tabelle 3.3: Übersicht der nicht-funktionalen Anforderungen mit ID, Anforderungsbeschreibung, Prioritätseinstufung und Zuordnung zu Qualitätsmerkmalen gemäß ISO/IEC 25010.

ID	**Formulierung der Anforderung**	**Priorität**	**Qualitäts-merkmal**
NFA01	Die App SOLLTE in der Lage sein, auf Android- und iOS-Systemen zu funktionieren.	Could-have	
NFA02	Das System MUSS dem Nutzer eine Fehlermeldung anzeigen, wenn bei der OCR-Verarbeitung kein Text erkannt wurde.	Must-have	Usability
NFA03	Das System MUSS dem Nutzer eine Fehlermeldung anzeigen, wenn der Speichervorgang fehlschlägt.	Must-have	Usability
NFA04	Das System MUSS dem Nutzer eine Fehlermeldung anzeigen, wenn der PDF-Export fehlschlägt.	Must-have	Usability
NFA05	Das System MUSS dem Nutzer eine visuelle oder textuelle Rückmeldung geben, wenn ein Prozess gestartet wurde (z.B. durch Ladeindikator oder Meldung).	Must-have	Usability
NFA06	Die Schriftgrößen im System MÜSSEN sich an den üblichen Standards mobiler Anwendungen orientieren: 11–16 px für normalen Text, 14–22 px für Überschriften.	Should-have	Usability
NFA07	Alle Texte MÜSSEN ein Kontrastverhältnis von mindestens 4,5:1 (normaler Text) bzw. 3:1 (großer Text, $\geq 18\,\text{pt}$ oder $\geq 14\,\text{pt}$ fett) zum Hintergrund aufweisen, gemäß WCAG 2.2, Erfolgskriterium 1.4.3.[3]	Must-have	Usability
NFA08	Interaktive Schaltflächen (Buttons) MÜSSEN ihren visuellen Zustand durch Farbänderung anzeigen, z.B. bei Aktivierung oder Deaktivierung.	Must-have	Usability
NFA09	Alle Buttons MÜSSEN sowohl ein beschreibendes Icon als auch eine textuelle Beschriftung enthalten.	Should-have	Usability
NFA10	Ressourcenintensive Prozesse wie Texterkennung (OCR), Furigana-Erzeugung, PDF-Export und serverbasierte Anfragen SOLLEN in der Regel nicht länger als 2 Sekunden dauern.	Should-have	Performance Efficiency
NFA11	Die für Texterkennung (OCR), Furigana-Erzeugung und Übersetzung verwendeten Modelle und APIs SOLLEN bevorzugt lokal auf dem mobilen Endgerät ausgeführt werden, um eine Offline-Nutzung zu ermöglichen und so Ressourcen effizient zu nutzen.	Should-have	Reliability, Security

Fortsetzung auf nächster Seite

[3] Accessibility Guidelines Working Group (AG WG) Participants (2025, 1. April). *WCAG 2.2: Contrast (Minimum) (Level AA)*. W3C. Abgerufen am 24. Juli 2025, von `https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html`

ID	Formulierung der Anforderung	Priorität	Qualitäts-merkmal
NFA12	Der Quellcode der Anwendung MUSS modular aufgebaut sein, um Übersichtlichkeit, Wartbarkeit und Erweiterbarkeit zu gewährleisten.	Must-have	Maintain-ability

3.6 Prototyping

Zur ersten Veranschaulichung der Benutzeroberfläche und des zugrunde liegenden Workflows wurden ein Mockup (siehe Abbildung 3.2) sowie ein Klickdummy der gesamten App erstellt. Das Mockup zeigt die zentralen Ansichten der Anwendung, darunter die Kameraansicht, das Texterkennungsergebnis mit Furigana und Übersetzung sowie die Übersicht über gespeicherte Dokumente.

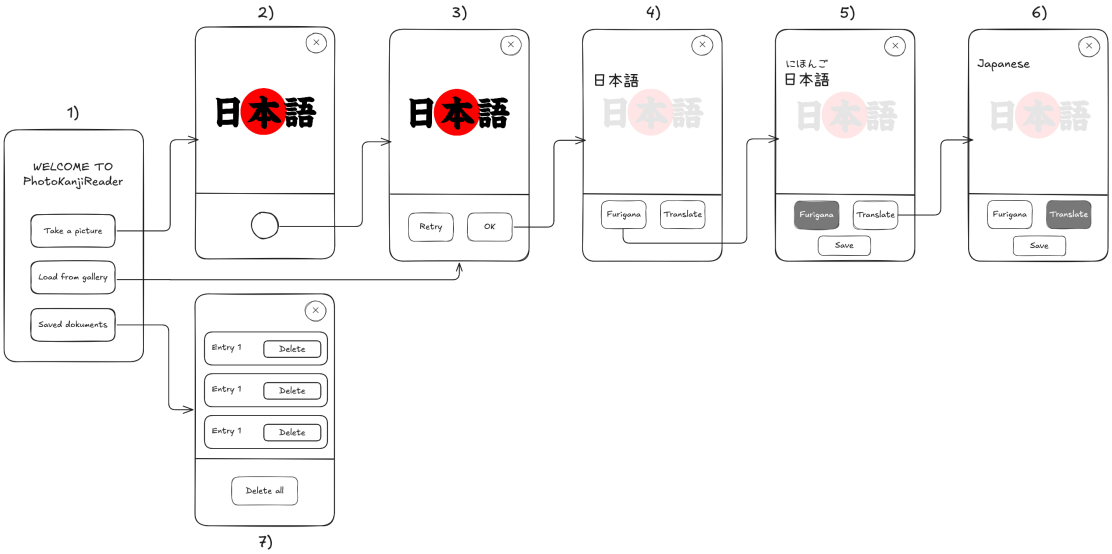


Abbildung 3.2: Mockup der Benutzeroberfläche. Die Pfeile symbolisieren den Wechsel zwischen den Bildschirmen. 1) Einstiegsbildschirm, 2) Kamera-Ansicht nach dem Öffnen der Kamera, 3) Ansicht nach Aufnahme eines Fotos oder Auswahl aus der Galerie, 4) Auswahlmöglichkeit: Anzeige von Furigana oder Übersetzung des erkannten Textes, 5) Darstellung im Furigana-Modus, 6) Darstellung im Übersetzungsmodus, 7) Übersicht über alle gespeicherten Einträge.

Der Post-OCR-Screen, dargestellt in Bildschirm 4, bildet die zentralen Funktionen der App ab: Furigana- und Übersetzungsdarstellung. Damit gehört er zu den wichtigsten Interaktionspunkten der Anwendung. Abbildung 3.3 zeigt mehrere Varianten dieses Screens, um eine gestalterische und funktionale Bewertung der Benutzeroberfläche zu ermöglichen.

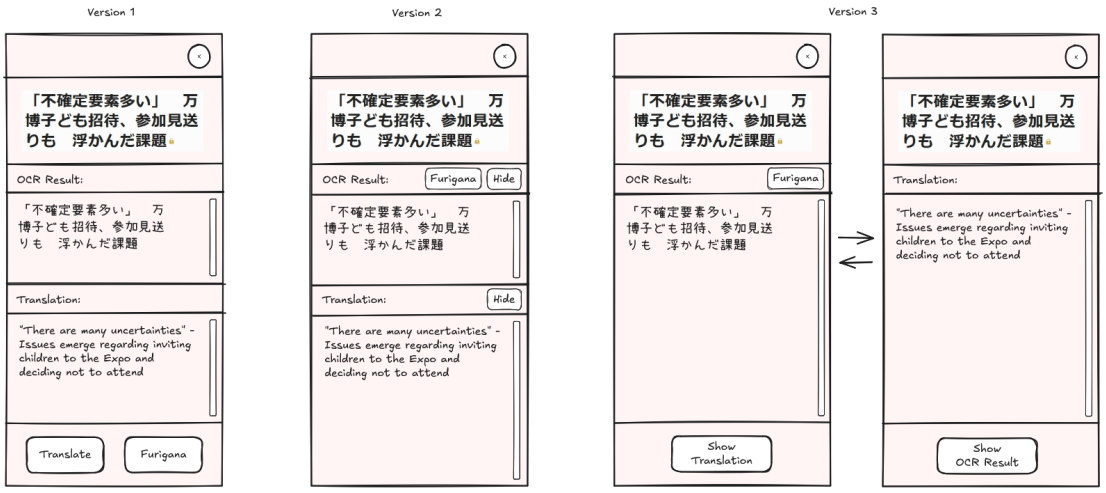


Abbildung 3.3: Vergleich mehrerer Interface-Varianten des Post-OCR-Screens zur Analyse von Benutzerführung und Funktionsdarstellung. Version 1 verwendet Toggle-Buttons, um zwischen Furigana- und Übersetzungsanzeige zu wechseln. Version 2 zeigt das OCR-Ergebnis, Furigana und die Übersetzung gemeinsam auf einem Bildschirm an. In Version 3 wechselt der Nutzer aktiv zwischen einem Furigana- und einem Übersetzungsbildschirm.

4

Technische Konzeption und Implementierung

Zu Beginn der technischen Umsetzung stand die Wahl eines geeigneten Frameworks für die plattformübergreifende Entwicklung. Eine besondere Herausforderung stellte die Suche nach lokal ausführbaren, speicherschonenden Bibliotheken dar. Hinzu kam die sprachliche Ausrichtung auf Japanisch, für die nur begrenzte technische Ressourcen und spezialisierte Technologien verfügbar sind. Dieses Kapitel stellt die zentralen technischen Entscheidungen des Projekts sowie deren praktische Umsetzung vor.

4.1 Frameworks

Frameworks erleichtern die Entwicklung von Anwendungen, indem sie wiederverwendbare Komponenten und unterstützende Funktionen bereitstellen. Zur Umsetzung der App kamen das Framework React Native sowie das darauf aufbauende Toolset Expo zum Einsatz. Nachfolgend werden beide Technologien näher vorgestellt.

4.1.1 React Native

Für die plattformübergreifende Entwicklung der mobilen Anwendung wurde das Open-Source-Framework *React Native* verwendet.[1] Es basiert auf der JavaScript-Bibliothek *React*, die ursprünglich von Meta zur Entwicklung interaktiver Benutzeroberflächen für das Web entwickelt wurde. React verwendet ein komponentenbasiertes Architekturmodell, bei dem die Benutzeroberfläche aus wiederverwendbaren Modulen zusammengesetzt wird. Dieses Prinzip übernimmt auch React Native.
Mit React Native können Entwickler mobile Anwendungen mit JavaScript oder TypeScript umsetzen und dabei eine gemeinsame Codebasis für Android- und iOS-Geräte verwenden. Anders als bei klassischen Cross-Plattform-Ansätzen wird hierbei jedoch keine Webansicht genutzt. Stattdessen werden die React-Komponenten zur Laufzeit in echte native UI-Elemente übersetzt –auf Android etwa in `View`-Komponenten aus Java oder Kotlin, auf iOS in `UIView`-Elemente aus Swift oder Objective-C.

[1] O. V. (o. D.). React Native. Abgerufen am 30. Juli 2025, von `https://reactnative.dev/`

Die App-Logik wird in einer JavaScript-Engine (z. B. Hermes oder JSC) ausgeführt und über eine sogenannte *Bridge* mit den nativen Plattformmodulen verbunden. Dadurch lässt sich mit Web-Technologien eine App entwickeln, die sich hinsichtlich Bedienung und Performance wie eine native Anwendung verhält.

4.1.2 Expo

Zur Unterstützung der Entwicklung wurde das Open-Source-Framework *Expo* eingesetzt.[2] Expo baut auf React Native auf und bietet eine erweiterte Entwicklungsumgebung, die viele Aufgaben während der App-Entwicklung vereinfacht. Dazu gehören unter anderem der Zugriff auf native Funktionen wie Kamera, Dateisystem und Lokalspeicher, ohne dass zusätzliche native Konfigurationen notwendig sind.
Ein besonderer Vorteil von Expo liegt darin, dass viele häufig genutzte Module bereits integriert sind und somit plattformübergreifend zuverlässig funktionieren. Darüber hinaus vereinfacht Expo den Build-Prozess sowie das Testen auf realen Geräten erheblich.
Zur Implementierung der Anwendung wurde *TypeScript* als Programmiersprache verwendet. Expo unterstützt TypeScript vollständig und ermöglicht dadurch eine stabile und gut wartbare Codebasis.

4.2 Verzeichnisstruktur

Die Verzeichnisstruktur der App basiert auf dem von Expo vorgegebenen Standard, der beim Erstellen eines neuen Projekts mit `create-expo-app` automatisch angelegt wird. Dieses Werkzeug erzeugt mit dem Befehl `npx create-expo-app` ein startbereites Expo- und React Native-Projekt mit typischen Ordnern und Dateien wie `app`, `assets`, `components` oder `package.json`. Dabei nimmt das Tool einen Großteil der manuellen Einrichtung ab und sorgt so für eine klare und strukturierte Projektorganisation.[3]

Wie in Abbildung 4.1 zu sehen, ist das Hauptverzeichnis in mehrere Unterordner gegliedert, wobei eine klare Trennung der Funktionalitäten besteht: UI-bezogene Komponenten liegen in `app` und `components`, wohingegen `hooks` und `services` die funktionale Logik kapseln.

[2] O. V. (o. D.). Expo. Abgerufen am 30. Juli 2025, von `https://docs.expo.dev/`
[3] O. V. (o. D.). *Create a project*. Expo. Abgerufen am 30. Juli 2025, von `https://docs.expo.dev/get-started/create-a-project/`

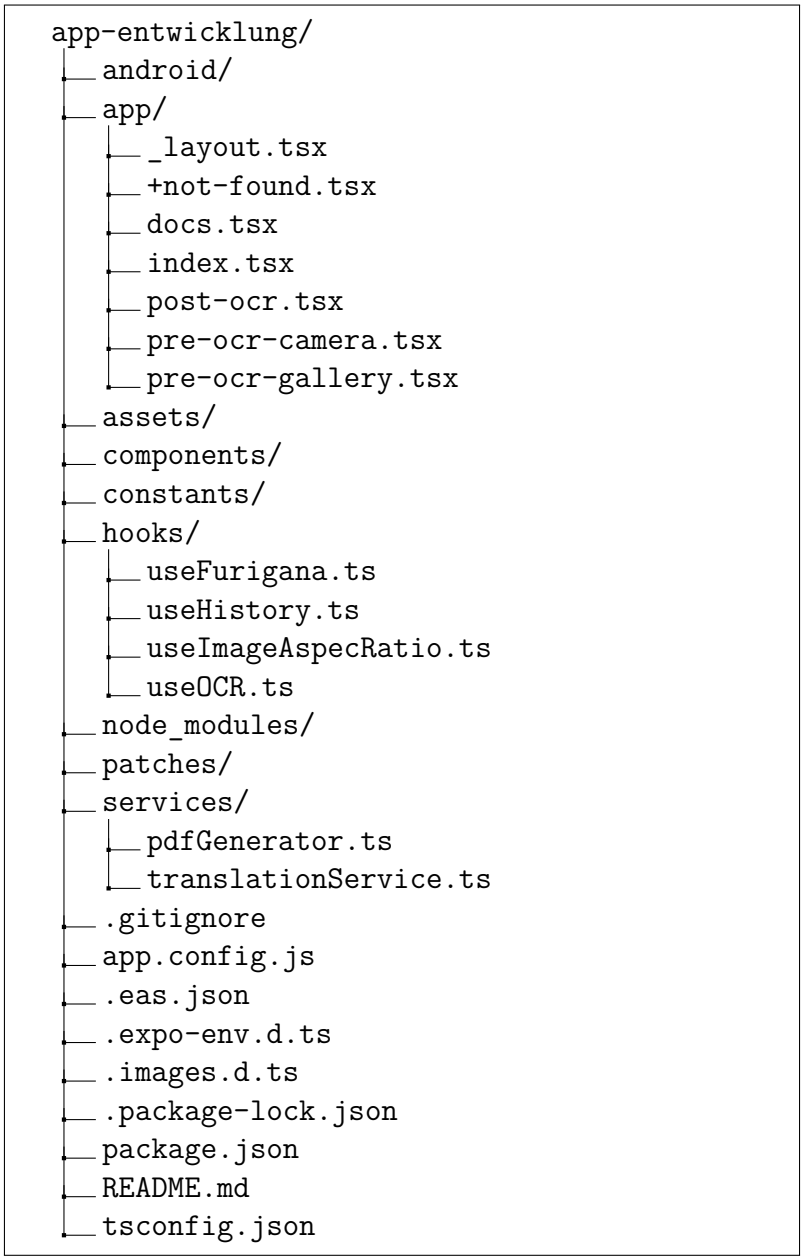


Abbildung 4.1: Verzeichnisstruktur der mobilen App

android/

Enthält nativen Code und Konfigurationsdateien für den Android-Build, also den Prozess, bei dem aus dem Quellcode eine lauffähige Android-App erstellt wird. Dieser Ordner ist nur im Bare Workflow oder nach Ausführen des Befehls `eject` vorhanden. Im Managed Workflow übernimmt Expo die vollständige Verwaltung der nativen Umgebung, was zwar die Entwicklung vereinfacht, jedoch die Integration nativer Erweiterungen einschränkt. In diesem Projekt wurde bewusst der Bare Workflow gewählt, um externe Bibliotheken einzubinden, die im Managed Workflow nicht unterstützt werden.

app/

Enthält ausschließlich plattformunabhängigen TypeScript-Code und keine nativen Komponenten. In diesem Verzeichnis befinden sich die Hauptbildschirmkomponenten sowie die zentrale Anwendungslogik der App.

assets/

Dient als Speicherort für statische Ressourcen wie Bilder und Schriftarten, die in der Benutzeroberfläche verwendet werden.

components/

Beinhaltet wiederverwendbare UI-Komponenten wie Footer oder Buttons, die in mehreren Screens zum Einsatz kommen.

constants/

Enthält zentral verwaltete Konstanten wie Farbpaletten, um Redundanz im Quellcode zu vermeiden und einheitliche Werte sicherzustellen.

hooks/

Beinhaltet benutzerdefinierte React Hooks zur Kapselung spezifischer Logik, beispielsweise zur Texterkennung oder zur Erzeugung von Furigana.

node_modules/

Enthält die automatisch installierten Abhängigkeiten des Projekts. Dieses Verzeichnis wird von `npm` verwaltet und sollte nicht manuell verändert werden.

patches/

Beinhaltet manuelle Anpassungen (Patches) an einzelnen `npm`-Paketen, um gezielt Fehler oder Kompatibilitätsprobleme zu beheben.

services/

Enthält Dienste, die externe Funktionalitäten kapseln, etwa die Anbindung an Übersetzungs-APIs (`translationService.ts`) oder die PDF-Generierung (`pdfGenerator.ts`).

app.config.js

Definiert die Konfiguration der App für Expo. Diese Datei enthält wichtige Metadaten und Einstellungen wie Name, Version, Icons, Splash Screen, Berechtigungen sowie Build-spezifische Konfigurationen.

.eas.json

Konfiguriert die Build-Profile für Expo Application Services (EAS). Sie legt fest, wie die App für Entwicklung, Vorschau und Produktion gebaut und veröffentlicht wird. EAS ist ein Cloud-Service von Expo, der native Builds, App-Store-Uploads und Updates automatisiert.

.expo-env.d.ts

Bindet automatisch Expo-Typdefinitionen in das TypeScript-Projekt ein und ermöglicht Typprüfung sowie Autovervollständigung. Sie beschreibt die Struktur und Verwendung Expo-spezifischer Funktionen und Komponenten.

.images.d.ts

Enthält Modul-Deklarationen für Bilddateien (z. B. `.png`, `.jpg`) und erlaubt deren typisierten Import in TypeScript-Projekten.

.package-lock.json

Hält die exakten Versionen aller Abhängigkeiten fest und gewährleistet damit reproduzierbare Installationen über verschiedene Systeme hinweg.

package.json

Ist die zentrale Konfigurationsdatei des Projekts. Sie enthält Metadaten wie Projektname, Version und Skripte sowie eine Übersicht aller Abhängigkeiten und deren Versionen, die für den Betrieb und die Entwicklung der Anwendung benötigt werden.

tsconfig.json

Definiert die Einstellungen für den TypeScript-Compiler, einschließlich der einzubeziehenden Dateien und der Regeln für deren Verarbeitung.

4.3 Funktionale Komponenten

Die App setzt sich aus mehreren funktionalen Komponenten zusammen, die den gesamten Prozess von der Bilderfassung bis zum PDF-Export abbilden. Abbildung 4.2 bietet einen Überblick über die einzelnen Funktionseinheiten, die in den folgenden Abschnitten näher erläutert werden.

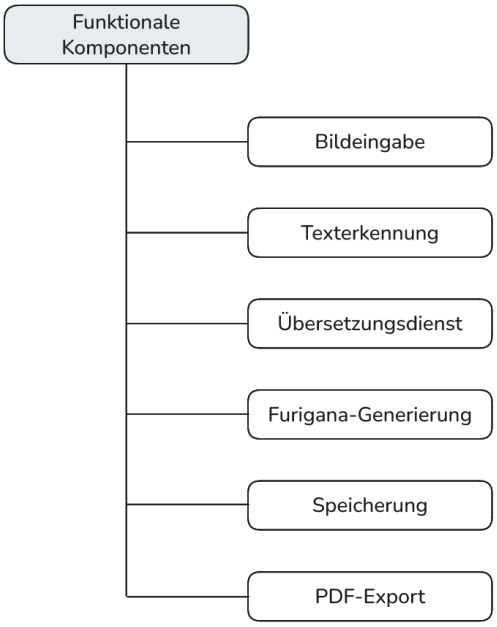


Abbildung 4.2: Überblick über die Hauptfunktionen der App. Jede Komponente übernimmt eine klar abgegrenzte Aufgabe im Verarbeitungsprozess.

4.3.1 Bildimport-Komponenten (Kamera und Galerie)

Um dem Benutzer das Hochladen eines Bildes zur anschließenden Texterkennung zu ermöglichen, bietet die Anwendung zwei Optionen: das direkte Aufnehmen eines Fotos mit der Gerätekamera oder das Auswählen eines vorhandenen Bildes aus der Fotogalerie. Beide Funktionen werden mithilfe der Bibliothek `expo-image-picker` umgesetzt, die eine einheitliche Schnittstelle für den Zugriff auf Kamera und Medienbibliothek in React Native bereitstellt.

Implementierung

Die Bildaufnahme erfolgt über die Funktion `takePhoto`. Nach erfolgreicher Berechtigungsanfrage durch `getPermission` wird mit `launchCameraAsync` die Rückkamera geöffnet (`cameraType: ImagePicker.CameraType.back`). Durch die Einstellung `allowsEditing: true` kann der Benutzer das Bild nach der Aufnahme zuschneiden bzw. bearbeiten. Wenn der Benutzer den Vorgang nicht abbricht, wird die URI des aufgenommenen Bildes aus dem Ergebnis extrahiert und mithilfe der Funktion `setImage` im Zustand der Komponente gespeichert. Diese Funktion stammt vom React-Hook `useState` und stellt sicher, dass das Bild intern verfügbar ist, zum Beispiel für die anschließende Texterkennung. Der Zugriff auf die Fotogalerie erfolgt analog über die Funktion `pickImage`, bei der anstelle von `launchCameraAsync` die Methode `launchImageLibraryAsync` verwendet wird, um ein vorhandenes Bild aus der Medienbibliothek auszuwählen. Auch hier wird die Bild-URI im Zustand gespeichert.

```
const takePhoto = async (setImage: (uri: string) => void): Promise<boolean | void
    ↪ > => {
    const hasPermission = await getPermission();
    if (!hasPermission) return;

    const result = await ImagePicker.launchCameraAsync({
        cameraType: ImagePicker.CameraType.back,
        allowsEditing: true,
        quality: 1,
    });

    if (!result.canceled) {
        setImage(result.assets[0].uri);
        return true;
    }
};
```

Listing 4.1: Bildaufnahme mit Kamera über `expo-image-picker`

4.3.2 Komponente zur Texterkennung (OCR)

Die optische Zeichenerkennung (engl. Optical Character Recognition, OCR) bezeichnet die automatisierte Erfassung und Umwandlung von gedruckten, getippten oder handschriftlichen Texten aus Bildquellen in maschinenlesbare Zeichen. Dabei kann es sich um gescannte Dokumente, PDF-Dateien oder digitale Fotografien handeln[JCI14, S.1].
Im nachfolgenden Abschnitt wird auf die Komponente zur Texterkennung eingegangen. Die Qualität des OCR-Ergebnisses ist entscheidend, da die Komponenten zur Furigana-Generierung und Übersetzung darauf aufbauen.

Verwendete Technologien

Für die Implementierung der OCR-Funktionalität wurde das npm-Paket `@react-native-ml-kit/text-recognition` verwendet. Dieses Paket fungiert als Wrapper für die *Google ML Kit Text Recognition API*, die ursprünglich in den nativen Programmiersprachen Java oder Kotlin für Android sowie Swift oder Objective-C für iOS bereitgestellt wird. Der Wrapper macht die Texterkennungsfunktion von ML Kit in React Native verfügbar, ohne dass plattformspezifischer nativer Code geschrieben werden muss. Die Texterkennung findet vollständig lokal auf dem Gerät statt (On-Device), wodurch keine Daten an externe Server gesendet werden.

Implementierung

Kernstück der OCR-Logik sind zwei Komponenten: die Funktion `performOCR`, welche die eigentliche Texterkennung ausführt, sowie der React-Hook `useOCR`, der diesen Prozess automatisiert und das Ergebnis verwaltet.

performOCR: Texterkennung

Die Funktion `performOCR` kapselt die Logik zur Texterkennung und erwartet als Eingabe die URI eines Bildes, die auf den Speicherort der entsprechenden Bilddatei verweist. Über die Methode `TextRecognition.recognize` erfolgt der Zugriff auf das OCR-Modell von ML Kit. In der Konfiguration wurde die Verwendung des `TextRecognitionScript.JAPANESE` aktiviert, um speziell auf die Erkennung japanischer Schriftzeichen optimiert zu sein. Standardmäßig werden jedoch auch lateinische Buchstaben erkannt. Nach erfolgreicher Texterkennung liefert die API ein Objekt zurück, das neben dem gesamten erkannten Text als String auch eine strukturierte Liste von Textblöcken (`blocks`) enthält. Jeder Block beinhaltet wiederum eine Liste von Zeilen (`lines`). Sowohl jeder Block als auch jede Zeile verfügen über Rahmendaten (`frame`), die Position (`top`, `left`) und Größe (`width`, `height`) beschreiben. Diese Informationen sind wichtig, um später die Schreibrichtung zu bestimmen und die Texteinheiten bei vertikalem Text richtig zu sortieren. Abschließend werden die Zeilen zu einem Textstring kombiniert und zurückgegeben.

```
1  const performOCR = async (imageUri: string, onEmptyText?: () => void): Promise<
       ↪ string> => {
2      if (!imageUri || typeof imageUri !== 'string') return "Invalid image format
           ↪ .";

4      try {
5          const result = await TextRecognition.recognize(imageUri,
               ↪ TextRecognitionScript.JAPANESE);
6          const blocks: OCRBlock[] = result.blocks ?? [];

8          if (!blocks.length) {
9              onEmptyText?.();
10             return "No Text Detected";
11         }

13         const vertical = isVerticalLayout(blocks);
14         const sortedBlocks = sortBlocks(blocks, vertical);

16         const rawText = sortedBlocks.map((block) => {
17             const sortedLines = sortLines(block.lines ?? [], vertical);
18             return sortedLines.map(line => line.text.trim()).join('\n');
19         }).join('\n');

21         return rawText || "No Text Detected";

23     } catch (error) {
24         return "Text Recognition Error";
25     }
26 };
```

Listing 4.2: Funktion `performOCR` zur Texterkennung

Verarbeitung vertikal angeordneter Texte

Problematisch gestaltete sich die Erkennung von vertikalen Texten mithilfe der Google ML Kit API. Beobachtungen ergaben, dass das OCR-Modell vertikale Texte in der westlichen Leserichtung von links nach rechts erkennt. Im Japanischen werden vertikale Texte jedoch primär von rechts nach links geschrieben. Ohne

weitere Verarbeitung führt das folglich zu falschen Texterkennungsergebnissen.

Ein möglicher Lösungsansatz für dieses Problem nutzt die geometrischen Informationen des erkannten Textes. Wie in Abbildung 4.3 dargestellt, unterteilt die API den Text in Blöcke und Linien und liefert dabei unter anderem die Rahmendaten zurück.[4]

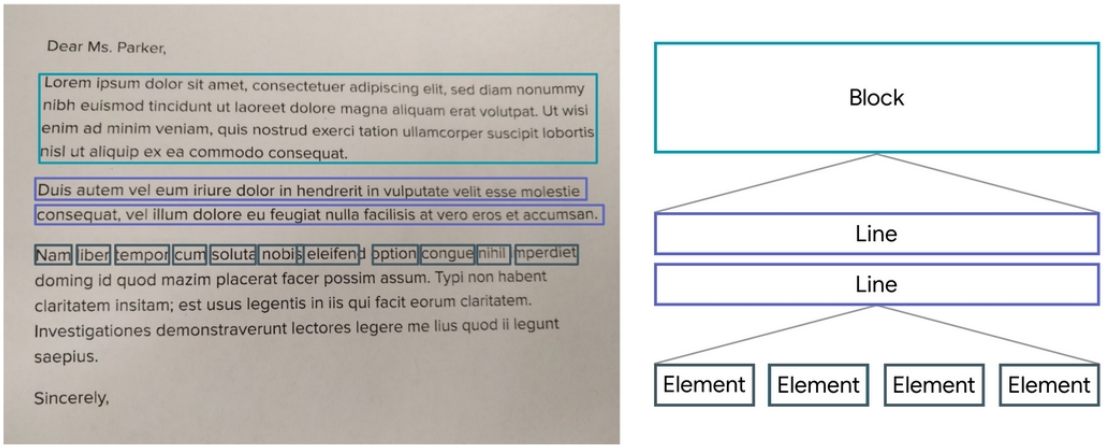


Abbildung 4.3: Hierarchische Struktur der Texterkennung in ML Kit: Der Text wird in Blöcke, Zeilen und Wörter unterteilt.

Indem die Breite und Höhe der Rahmen miteinander verglichen werden, lässt sich die Ausrichtung einzelner Blöcke bestimmen. Dabei wird ein Text als vertikal klassifiziert, wenn bei allen Blöcken die Höhe größer ist als die Breite.

```
1  const isVerticalLayout = (blocks: OCRBlock[]) => {
2      for (const block of blocks) {
3          const height = block.frame?.height ?? 0;
4          const width = block.frame?.width ?? 0;
5          if (height <= width) {
6              return false;
7          }
8      }
9      return true;
10 };
```

Listing 4.3: Funktion `isVerticalLayout` zur Erkennung vertikaler Textrichtung, aufgerufen innerhalb von `performOCR`

Wenn `isVerticalLayout` `true` zurückgibt und der Text somit als vertikal erkannt wurde, werden die Textblöcke und die darin enthaltenen Zeilen entsprechend umsortiert.

```
1  const sortBlocks = (blocks: OCRBlock[], vertical: boolean) => {
2      if (!vertical) {
```

[4] O. V. (o. D.). *Texterkennung Version 2*. Google. Abgerufen am 11. Juli 2025, von `https://developers.google.com/ml-kit/vision/text-recognition/v2?hl=de`

```
3           return blocks;
4       }
5       return [...blocks].sort((a, b) => {
6           const aFrame = a.frame ?? { top: 0, left: 0, width: 0, height: 0 };
7           const bFrame = b.frame ?? { top: 0, left: 0, width: 0, height: 0 };
8           return bFrame.left - aFrame.left;
9       });
10  };
```

Listing 4.4: Funktion zur Sortierung der Textblöcke, aufgerufen in `performOCR`

```
1   const sortLines = (lines: OCRLine[], vertical: boolean) => {
2       if (!vertical) {
3           return lines;
4       }
5       return [...lines].sort((a, b) => {
6           const aFrame = a.frame ?? { top: 0, left: 0, width: 0, height: 0 };
7           const bFrame = b.frame ?? { top: 0, left: 0, width: 0, height: 0 };
8           return bFrame.left - aFrame.left;
9       });
10  };
```

Listing 4.5: Funktion zur Sortierung der Textzeilen, aufgerufen in `performOCR`

Abbildung 4.4 illustriert exemplarisch, wie ein vertikaler Text vom OCR-Modell erkannt wird und wie sich die Textstruktur nach der anschließenden Umsortierung darstellt.

=== Vor der Sortierung ===
Block 1:
Zeile 1: たのて本状を投模します
Zeile 2: 和えた高度な指圧療法をはれ、
Zeile 3: 経絡治療とき本独自の投護術を
Zeile 4: 右者医王会に沙て東洋古来の、
Block 2:
Zeile 1: 極了註

=== Nach der Sortierung ===
Block 1:
Zeile 1: 極了註
Block 2:
Zeile 1: 右者医王会に沙て東洋古来の、
Zeile 2: 経絡治療とき本独自の投護術を
Zeile 3: 和えた高度な指圧療法をはれ、
Zeile 4: たのて本状を投模します

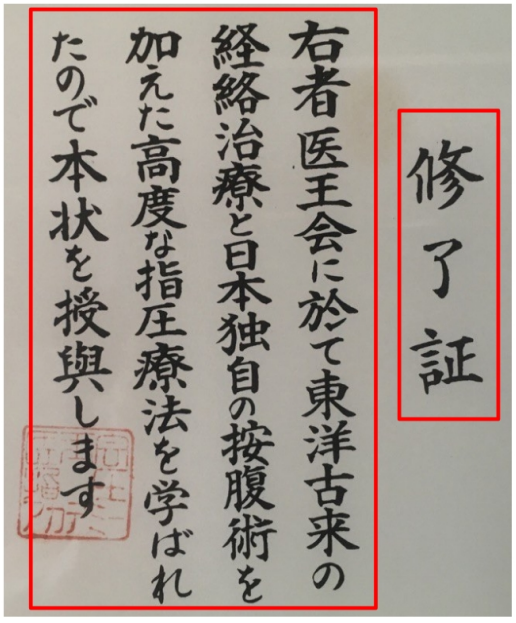


Abbildung 4.4: Rechts ist das Bild zu sehen, auf das die OCR angewendet wurde. Die roten Kästen zeigen die Textblöcke, wie sie die API erkennt. Oben links ist die von der API erfasste Textstruktur dargestellt. Unten links ist die Textstruktur nach der Umsortierung der Blöcke und Zeilen entsprechend der japanischen Leserichtung für vertikale Texte (von rechts nach links) zu sehen.

useOCR: Einbindung der OCR-Funktionalität in die Benutzeroberfläche

Die Texterkennung wird über den benutzerdefinierten React-Hook `useOCR` in die Anwendungslogik integriert. Sobald ein Bild übergeben wird, startet der Hook automatisch die OCR-Verarbeitung durch Aufruf der Funktion `performOCR`. Dabei verwaltet er zwei zentrale Zustände mittels des React-`useState`-Hooks: Zum einen `ocrText`, der den erkannten und vorverarbeiteten Text als String speichert, zum anderen `ocrLoading`, einen booleschen Wert, der den aktuellen Ladezustand des OCR-Prozesses abbildet und es der Benutzeroberfläche ermöglicht, z. B. Ladeanzeigen oder Spinner darzustellen.
Wird kein Bild übergeben, weist der Hook dem Zustand `ocrText` eine entsprechende Fehlermeldung zu. Sollte im Bild kein Text erkannt werden, informiert eine Alert-Meldung den Nutzer über den Fehler und navigiert ihn automatisch zurück zur Startansicht der Anwendung. Die Navigation erfolgt über das `router`-Objekt, das durch den Hook `useRouter()` bereitgestellt wird und das Wechseln zwischen verschiedenen Seiten innerhalb der Anwendung ermöglicht.
Der erkannte Text wird vor dem Speichern bereinigt und anschließend im Zustand `ocrText` abgelegt, sodass er unmittelbar in der Benutzeroberfläche weiterverwendet werden kann.

```
1  export const useOCR = (image?: string) => {
2      const [ocrText, setOcrText] = useState('');
3      const [ocrLoading, setOcrLoading] = useState(false);
4      const router = useRouter();
5
6      useEffect(() => {
7          const processImage = async () => {
8              if (!image) {
9                  setOcrText("No image passed.");
10                 return;
11             }
12
13             setOcrLoading(true);
14             const rawText = await performOCR(image, () =>
15             Alert.alert('Oops!', 'No text found. Try another image.', [{ text: '
                   ↪ Got it', onPress: () => router.dismissTo('/') }])
16             );
17             setOcrText(cleanRecognizedText(rawText));
18             setOcrLoading(false);
19         };
20         processImage();
21     }, []);
22
23     return { ocrText, setOcrText, ocrLoading };
24 };
```

Listing 4.6: React Hook `useOCR` zur Texterkennung

Post-Processing: Entfernung unerwünschter Zeilenumbrüche

Die OCR-API teilt Text mitunter unzutreffend in Blöcke auf, wodurch ungewollte Zeilenumbrüche entstehen und der inhaltliche Zusammenhang gestört wird. Dies wirkt sich nachteilig auf nachgelagerte Verarbeitungsschritte wie etwa Übersetzungen aus. Um dieses Problem zu beheben, entfernt die Funktion

`cleanRecognizedText` automatisch generierte Umbrüche aus dem Rohtext. Dabei werden ausschließlich solche Zeilenumbrüche entfernt, die nicht unmittelbar auf Satzzeichen wie Punkt, Fragezeichen oder japanische Satzendzeichen folgen. Die Umsetzung erfolgt mithilfe eines regulären Ausdrucks mit negativem Lookbehind.

```
1  const cleanRecognizedText = (text) => {
2      const sentencePunctuation = /[。！？!?.,]/;
3      if (sentencePunctuation.test(text)) {
4          return text.replace(/(?<![。！？!?.,])\n/g, '');
5      }
6      return text;
7  };
```

Listing 4.7: Bereinigung des erkannten Textes

4.3.3 Komponente zur Furigana-Erzeugung

Die Furigana-Funktion analysiert den per OCR erfassten Text und ergänzt Kanji automatisch mit Hiragana-Lesungen, die oberhalb der Zeichen in kleinerer Schrift eingeblendet werden.
Eine besondere Herausforderung bei der Implementierung bestand in der korrekten Verarbeitung aller möglichen Tokenformen. Im Japanischen setzen sich viele Wörter aus einer Kombination unterschiedlicher Schriftsysteme zusammen, wobei besonders die Mischung aus Kanji und Hiragana sehr häufig ist. Während Kanji in der Regel die bedeutungstragenden Elemente eines Wortes darstellen, übernehmen Hiragana vorwiegend grammatische Funktionen: Sie markieren Flexionen, beeinflussen die Lesung von Kanji oder dienen der Bildung spezieller Höflichkeitsformen.[5]
Da nur die Kanji-Anteile eines Tokens mit Furigana versehen werden sollen, ist eine präzise Segmentierung erforderlich, bei der die Lesung den jeweiligen Kanji korrekt zugeordnet wird. Dabei ist zu berücksichtigen, dass Hiragana nicht nur vor oder nach, sondern auch zwischen Kanji-Zeichen auftreten kann. Abbildung 4.5 zeigt Beispiele für Token mit unterschiedlich positionierten Hiragana-Anteilen und deren Umsetzung in Furigana-Schreibweise.

[5] O. V. (o. D.). *Okurigana*. Wikipedia. Abgerufen am 10. Juli 2025, von `https://en.wikipedia.org/wiki/Okurigana`

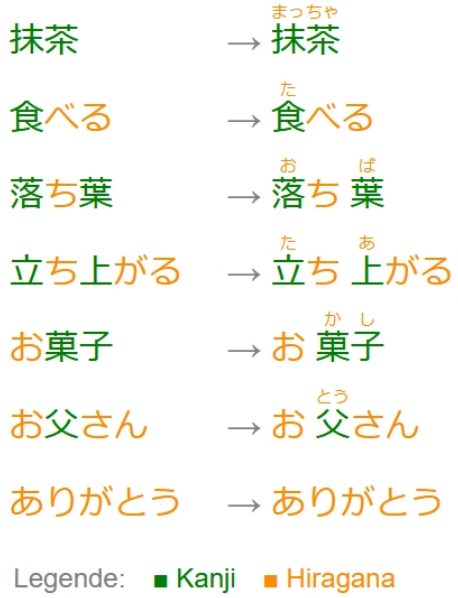


Abbildung 4.5: Beispiele für Kanji- und Hiragana-Bestandteile in Token und deren Umsetzung in Furigana-Schreibweise.

Verwendete Technologien

Das Herzstück der Furigana-Erzeugung ist die Java-Bibliothek *Kuromoji*, die speziell für die Tokenisierung und linguistische Analyse japanischer Texte entwickelt wurde.[6] Sie basiert auf integrierten Wörterbüchern wie IPADIC, die detaillierte Informationen zu Wortformen, Lesungen und grammatikalischen Merkmalen enthalten. In dieser App wird Kuromoji über die React-Native-Bridge `react-native-japanese-text-analyzer` eingebunden. Die Integration ermöglicht eine vollständig lokale Verarbeitung, ohne Internetverbindung oder Serverkommunikation.[7]
Nach der Texterkennung durch OCR wird der erfasste Text mit Kuromoji in einzelne Token zerlegt. Für jedes Token liefert die Bibliothek unter anderem die ursprüngliche Wortform (*surface_form*) sowie die Lesung (*reading*) in Katakana. Diese Lesung bildet die Grundlage für die spätere Anzeige von Furigana über den Kanji-Zeichen. Ein Beispiel der erzeugten Analyse-Daten zeigt Abbildung 4.6.
Zur Umwandlung der Katakana-Lesungen in das für Furigana übliche Hiragana kommt zusätzlich die JavaScript-Bibliothek `wanakana` zum Einsatz. Sie bietet eine schlanke API zur Transkription zwischen Hiragana, Katakana und Romaji.

[6] O. V. (o. D.). *kuromoji*. atilika. Abgerufen am 10. Juli 2025, von `https://www.atilika.com/en/kuromoji/`
[7] Kidd, S. (o. D.). *swkidd/react-native-japanese-text-analyzer*. GitHub. Abgerufen am 10. Juli 2025, von `https://github.com/swkidd/react-native-japanese-text-analyzer`

```
[ {
    surface_form: '黒文字',     // 表層形
    pos: '名詞',                // 品詞
    pos_detail_1: '一般',       // 品詞細分類1
    pos_detail_2: '*',          // 品詞細分類2
    pos_detail_3: '*',          // 品詞細分類3
    conjugated_type: '*',       // 活用型
    conjugated_form: '*',       // 活用形
    basic_form: '黒文字',       // 基本形
    reading: 'クロモジ',        // 読み
    pronunciation: 'クロモジ'   // 発音
  } ]
```

Abbildung 4.6: Analyseergebnisse von Kuromoji. Für die Furigana-Generierung relevant sind insbesondere die Felder `surface_form` (ursprüngliche Wortform) und `reading` (Lesung in Katakana).[7]

Implementierung

Die Erzeugung von Furigana basiert auf einer Kombination aus Tokenisierung mit Kuromoji und der anschließenden Verarbeitung in TypeScript. Dabei werden den einzelnen Kanji automatisch die passenden Lesungen zugeordnet. Die dafür entwickelte Logik ist in modular aufgebaute Funktionen gegliedert, die im Folgenden im Detail erläutert werden.

useFurigana: Integration und Steuerung der Furigana-Analyse

Die Funktion `useFurigana` ist als React-Hook umgesetzt und dient als zentrale Schnittstelle zur Integration der Furigana-Analyse in die Anwendung. Sie verwaltet zwei lokale Zustände: `furiganaBlocks` speichert das Analyseergebnis in Form von Arrays strukturierter Textfragmente, während `isLoading` anzeigt, ob gerade ein Analyseprozess läuft.
Die Methode `triggerFurigana` prüft zunächst, ob ein Eingabetext vorliegt und ob bereits eine Analyse im Gange ist. Ist beides nicht der Fall, wird die Funktion `generateFurigana` asynchron aufgerufen. Das Ergebnis wird in den Zustand `furiganaBlocks` übernommen. Bei einem Fehler wird dieser protokolliert und der Zustand zurückgesetzt.

```
1  export const useFurigana = () => {
2      const [furiganaBlocks, setFuriganaBlocks] = useState<FuriganaItem[][]|null>;
3      const [isLoading, setIsLoading] = useState(false);

5      const triggerFurigana = async (ocrText: string) => {
6          if (!ocrText || isLoading) return;
7          setIsLoading(true);
8          try {
9              const blocks = await generateFurigana(ocrText);
10             setFuriganaBlocks(blocks);
11         } catch (error) {
```

```
12              console.error('Furigana-Fehler: ', error);
13              setFuriganaBlocks(null);
14          } finally {
15              setIsLoading(false);
16          }
17      };
18      return { furiganaBlocks, triggerFurigana, setFuriganaBlocks, isLoading };
19  };
```

Listing 4.8: React-Hook zur Integration der Furigana-Funktion in die Benutzeroberfläche

generateFurigana: Textaufbereitung und Tokenisierung

Die Funktion `generateFurigana` analysiert den Eingabetext asynchron und erzeugt strukturierte Textsegmente mit optionalen Furigana. Enthält der Text keine Kanji, wird `null` zurückgegeben. Andernfalls wird er zunächst mit `splitIntoBlocks` in kleinere Abschnitte unterteilt. Die anschließende Tokenisierung erfolgt blockweise über die Kuromoji-Funktion `tokenize`, die zu jedem erkannten Token die ursprüngliche Wortform (`surface_form`) sowie die zugehörige Katakana-Lesung (`reading`) liefert. Diese Lesung wird anschließend mit `toHiragana` in das für Furigana gebräuchliche Hiragana-Schriftsystem überführt. Danach werden Token und Lesung mithilfe von `splitToken` in kleinere Einheiten aufgeteilt, sodass jedem Kanji die passende Lesung zugewiesen werden kann.

```
1  const generateFurigana = async (ocrText: string): Promise<FuriganaItem[][]|null>
       ↪ => {
2      if (!containsKanji(ocrText)) {
3          return null;
4      }
5      const textBlocks = splitIntoBlocks(ocrText);
6      const processedBlocks = await Promise.all(
7          textBlocks.map(async (block) => {
8              const tokens: Token[] = await tokenize(block);
9              return tokens.flatMap(({ surface_form, reading }) =>
10                 splitToken(surface_form, toHiragana(reading))
11             );
12         })
13     );
14     return processedBlocks;
15 };
```

Listing 4.9: Funktion zur Analyse eines Eingabetexts, aufgerufen in `useFurigana`

Das Ergebnis ist ein verschachteltes Array vom Typ `FuriganaItem[][]`, wobei jede äußere Liste einem Textblock entspricht. Die inneren Listen enthalten `FuriganaItem`-Objekte mit einem Textfragment und optionaler Lesung.
Die zugrunde liegenden Typen sind in Listing 4.10 dargestellt.

```
1  type FuriganaItem = {
2      text: string;
3      furigana?: string;
4  };

6  type Token = {
7      surface_form: string;
```

```
8       reading?: string;
9   };
```

Listing 4.10: Struktur der Ausgabeobjekte

splitToken: Zerlegung eines Tokens in Furigana-Einheiten

Die Funktion `splitToken` verarbeitet ein Token bestehend aus Originaltext (`surface`) und optionaler Lesung (`reading`). Fehlt die Lesung oder enthält das Token keine Kanji, wird es als einfacher Textabschnitt ohne Furigana zurückgegeben. Andernfalls erfolgt eine Segmentierung in drei Teile: führender Text, Kanji-Kern und nachfolgender Text. Die Lesung des Kerns ergibt sich durch Entfernen der entsprechenden Silben aus der Gesamtlesung.
Enthält der Kanji-Kern selbst Hiragana, wird der eingebettete Hiragana-Block mithilfe von `findHiraganaChunk` isoliert. Anschließend werden Text und Lesung um diesen Block herum aufgeteilt und als strukturierte Liste von `FuriganaItem`s zurückgegeben.

```
1   const splitToken = (surface: string, reading?: string): FuriganaItem[] => {
2       if (!surface) return [];
3       if (!reading || !containsKanji(surface)) {
4           return [{ text: surface }];
5       }

7       const chars = [...surface];
8       const firstKanji = chars.findIndex(isKanji);
9       const lastKanji = chars.map(isKanji).lastIndexOf(true);

11      const leading = surface.slice(0, firstKanji);
12      const trailing = surface.slice(lastKanji + 1);
13      const kanjiCore = surface.slice(firstKanji, lastKanji + 1);
14      const readingCore = reading.slice(leading.length,
15      reading.length - trailing.length);

17      const items: FuriganaItem[] = [];
18      if (leading) items.push({ text: leading });

20      const hiraganaChunk = findHiraganaChunk(kanjiCore);
21      if (hiraganaChunk) {
22          const [kanji, kana] = splitCore(kanjiCore, readingCore, hiraganaChunk);

24          if (kanji.first) items.push({ text: kanji.first, furigana: kana.first });
25          items.push({ text: hiraganaChunk });
26          if (kanji.second) items.push({ text: kanji.second, furigana: kana.second
                ↪ });
27      } else {
28          items.push({ text: kanjiCore, furigana: readingCore });
29      }

31      if (trailing) items.push({ text: trailing });
32      return items;
33  };
```

Listing 4.11: Funktion zur Aufteilung eines Tokens in Text- und Furigana-Fragmente, aufgerufen in `generateFurigana`

splitCore: Auftrennung von Kanji-Kern und Lesung

Die Funktion `splitCore` wird aufgerufen, wenn der Kanji-Kern eines Tokens einen eingebetteten Hiragana-Block enthält. Sie bestimmt dessen Position im Text und segmentiert Kern und Lesung jeweils in drei Abschnitte: vorderer Teil, Hiragana-Block und nachfolgender Teil. Da der mittlere Abschnitt keine Furigana benötigt, gibt die Funktion nur die beiden Randbereiche zurück. Diese werden als zwei separate Objekte gebündelt: eines enthält die Kanji-Einheiten links und rechts des Hiragana-Blocks, das andere die entsprechenden Lesungen.

```
const splitCore = (
kanjiCore: string,
readingCore: string,
hiraganaChunk: string
): [{ first: string; second: string }, { first: string; second: string }] => {
    const start = kanjiCore.indexOf(hiraganaChunk);
    const end = start + hiraganaChunk.length;

    const rStart = readingCore.indexOf(hiraganaChunk);
    const rEnd = rStart + hiraganaChunk.length;

    return [
    { first: kanjiCore.slice(0, start), second: kanjiCore.slice(end) },
    { first: readingCore.slice(0, rStart), second: readingCore.slice(rEnd) },
    ];
};
```

Listing 4.12: Funktion zur Aufspaltung eines Kanji-Kerns mit eingebettetem Hiragana, aufgerufen in `splitToken`

Visuelle Darstellung von Furigana

Da React Native im Gegensatz zu HTML keine native Unterstützung für Ruby-Annotationen zur Darstellung von Lesungen über Kanji bietet, wird diese Funktionalität in der App durch eine eigene Komponente namens `Ruby` nachgebildet. Diese Komponente erhält ein Textfragment sowie optional eine Furigana-Lesung und zeigt beides gestapelt in einem gemeinsamen Container an: Die Lesung wird oberhalb des Kanji-Textes dargestellt. Damit entsteht eine visuelle Struktur, die der typischen Darstellung von Furigana im Japanischen sehr nahekommt.

```
export default function Ruby({ text, furigana }: Props) {
    return (
    <View style={styles.rubyContainer}>
        <Text style={styles.furigana}>{furigana ?? ' '}</Text>
        <Text style={styles.kanji}>{text}</Text>
    </View>
    );
}
```

Listing 4.13: Emulation von Ruby-Annotationen in React Native

Für die Gesamtausgabe der Analyseergebnisse ist die Funktion `renderFuriganaBlocks` zuständig. Sie durchläuft die verschachtelte Datenstruktur der `FuriganaItem`-Blöcke und erzeugt aus jedem Fragment ein

`Ruby`-Element. Auf diese Weise lassen sich sowohl einfache Textstellen als auch Kanji mit zugehöriger Lesung konsistent darstellen.

```
export function renderFuriganaBlocks(blocks: FuriganaItem[][]) {
    return blocks.map((block, index) => (
        <View key={index} style={styles.rows}>
        {block.map((item, idx) => (
            <Ruby key={idx} text={item.text} furigana={item.furigana} />
        ))}
        </View>
    ));
};
```

Listing 4.14: Rendering von Furigana-Blöcken mittels Ruby-Komponente

4.3.4 Übersetzungskomponente

Die Übersetzung erfolgt in der Anwendung über eine Zusammenarbeit zwischen Client und lokalem Server. Während der Server das eigentliche Sprachmodell ausführt, übernimmt die App die Ansteuerung und Verarbeitung der Übersetzungsanfragen auf der Clientseite.

Serverseitige Übersetzungslogik

Um eine automatische Übersetzung von japanischen Texten ins Englische zu ermöglichen, wurde ein lokaler Übersetzungsserver mit *Python* umgesetzt. Dieser basiert auf dem Framework *FastAPI* und verwendet ein vortrainiertes Modell von *Hugging Face*.

Verwendete Technologien

Für die Umsetzung des lokalen Übersetzungsservers wurden mehrere Python-Bibliotheken eingesetzt: Das Deep-Learning-Framework `torch` bildet die Grundlage für die Ausführung neuronaler Netze. `transformers` wird verwendet, um das vortrainierte Übersetzungsmodell von Hugging Face zu laden und anzuwenden. Die Schnittstelle zur mobilen Anwendung wurde mit `fastapi` realisiert, einem modernen Framework zur Erstellung asynchroner Web-APIs. Für die lokale Ausführung dient `uvicorn` als leichter und performanter ASGI-Server.

Für die Übersetzung von Japanisch nach Englisch wird das vortrainierte Modell `Helsinki-NLP/opus-mt-ja-en` verwendet, ein spezialisiertes MarianMT-Modell auf Basis der Transformer-Architektur. Das Modell ist mit dem öffentlichen OPUS-Datensatz trainiert worden und nutzt eine Kombination aus normalisierter Textvorverarbeitung und SentencePiece-Tokenisierung, um japanische Eingabetexte effizient zu verarbeiten.[8]
Das zugrunde liegende Framework basiert auf der MarianNMT-Architektur (in C++) und wurde für die Nutzung in PyTorch über die `transformers`-Bibliothek

[8] O. V. (o. D.). *Helsinki-NLP/opus-mt-ja-en*. Hugging Face. Abgerufen am 04.08.2025, von `https://huggingface.co/Helsinki-NLP/opus-mt-ja-en`

portiert.[9] Das Modell erzielt auf dem Tatoeba-Testset einen BLEU-Wert von ca. 41,7.[8]

Implementierung

Die Übersetzungs-API ist mit dem Webframework `FastAPI` umgesetzt und wird von einem lokalen `uvicorn`-Server bereitgestellt. Beim Start des Servers werden sowohl der Tokenizer als auch das vortrainierte Übersetzungsmodell `Helsinki-NLP/opus-mt-ja-en` geladen:

```
tokenizer = MarianTokenizer.from_pretrained(model_name)
model = MarianMTModel.from_pretrained(model_name)
```

Listing 4.15: Laden von Modell und Tokenizer beim Serverstart

Unter dem Pfad `/translate` stellt die API einen POST-Endpunkt bereit, an den die mobile Anwendung Texte zur Übersetzung senden kann. Die Daten werden dabei im JSON-Format im Anfragekörper übertragen.

```
class TranslationRequest(BaseModel):
    text: str
```

Listing 4.16: Definition des Anfragemodells für den POST-Endpunkt

```
@app.post("/translate")
def get_translation(request: TranslationRequest):
    output = translate(request.text)
    return {"translation": output}
```

Listing 4.17: FastAPI-Endpunkt zur Verarbeitung von Übersetzungsanfragen

Die eigentliche Übersetzungslogik ist in der Funktion `translate` umgesetzt: Der empfangene Text wird zunächst tokenisiert, dann vom Modell verarbeitet und schließlich in eine lesbare Zeichenkette überführt.

```
def translate(text):
    inputs = tokenizer(text, return_tensors="pt", padding=True)
    translated = model.generate(**inputs)
    result = tokenizer.decode(translated[0], skip_special_tokens=True)
    return { "translation": result }
```

Listing 4.18: Tokenisierung, Modellinferenz und Antwortformat

Der Server wird lokal mit folgendem Befehl gestartet:

```
uvicorn app:app --host 0.0.0.0 --port 8000
```

Listing 4.19: Startbefehl

[9] O. V. (o. D.). *Helsinki-NLP/Opus-MT*. GitHub. Abgerufen am 04.08.2025, von `https://github.com/Helsinki-NLP/Opus-MT`

Clientseitige Übersetzungslogik

Die Übersetzung erfolgt auf Client-Seite in der Methode `translateText`. Diese kommuniziert mit dem lokal laufenden Übersetzungsserver, übermittelt den Eingabetext zur Verarbeitung und bereitet die Antwort entsprechend auf. Der gesamte Ablauf gliedert sich in mehrere Schritte, die im Folgenden näher erläutert werden.

Implementierung

Zunächst wird der Eingabetext in einzelne Absätze unterteilt. Jeder dieser Absätze wird, sofern Satzzeichen vorhanden sind, mit der Funkion `splitAndGroupSentences` in Gruppen von maximal zwei Sätzen gegliedert. Diese Gruppierung dient der Qualitätssicherung, da sich bei zu langen Texteingaben in der Praxis eine deutliche Abnahme der Übersetzungsgenauigkeit zeigte.
Für die Verarbeitung werden zwei Arrays verwendet: `translatedGroups` sammelt die Übersetzungen der Satzgruppen innerhalb eines Blocks, während `finalTranslations` die blockweise zusammengesetzten Ergebnisse speichert.

```
1  export async function translateText(text: string): Promise<string> {
2      const finalTranslations: string[] = [];
3      const blocks = text.split('\n').map(block => block.trim());

5      for (const block of blocks) {
6          if (!block) {
7              finalTranslations.push('');
8              continue;
9          }
10         const translatedGroups: string[] = [];
11         const sentenceGroups = splitAndGroupSentences(block, 2);
```

Listing 4.20: Aufteilung in Blöcke und Satzgruppen

Für jede Satzgruppe wird eine POST-Anfrage an den lokalen Server gesendet. Falls ein Fehler auftritt (z. B. durch Verbindungsprobleme), wird der Vorgang nach einer kurzen Verzögerung einmalig wiederholt:

```
12         ...
13         for (const group of sentenceGroups) {
14             let success = false;
15             let translatedGroup = '';

17             for (let attempt = 1; attempt <= 2 && !success; attempt++) {
18                 try {
19                     const response = await fetch(LOCAL_API_URL, {
20                         method: 'POST',
21                         headers: { 'Content-Type': 'application/json' },
22                         body: JSON.stringify({ text: group }),
23                     });
24                     const result = await response.json();
25                     if (result?.translation) {
26                         translatedGroup = result.translation;
27                         success = true;
28                     }
29                 } catch (error) {
30                     // Fehlerbehandlung
31                 }
32                 if (!success) {
33                     await new Promise(resolve => setTimeout(resolve, 1000));
```

```
34                  }
35              }
```

Listing 4.21: Übersetzungsanfrage mit Retry

Nach Abschluss aller Anfragen werden die erfolgreich übersetzten Gruppen gesammelt. Gruppen, bei denen beide Übersetzungsversuche scheiterten, werden mit einem Platzhalter versehen. Abschließend erfolgt die Rekonstruktion des ursprünglichen Absatzlayouts:

```
36              ...
37              translatedGroups.push(translatedGroup || '[Translation Error]');
38          }
39          finalTranslations.push(translatedGroups.join(' '));
40      }
41      return finalTranslations.join('\n');
42  }
```

Listing 4.22: Zusammensetzen des finalen Ergebnisses

4.3.5 Speicherkomponente

Neben der Übersetzungskomponente bietet die Anwendung eine lokale Speicherfunktion, die OCR-Ergebnisse, Übersetzungen, Furigana-Daten sowie zugehörige Bilder dauerhaft speichert. Dadurch entsteht ein Übersetzungsverlauf, der auch bei späteren Sitzungen erhalten bleibt. Die Speicherung erfolgt vollständig clientseitig.

Verwendete Technologien

Zur Umsetzung der lokalen Speicherfunktionalität kommen zwei zentrale Bibliotheken zum Einsatz: Mit `@react-native-async-storage/async-storage` erfolgt die persistente Speicherung der Übersetzungsverläufe in Form von JSON-Daten. Für die Verwaltung lokaler Bilddateien wird das Modul `expo-file-system` verwendet, das den Zugriff auf das Dateisystem des Geräts ermöglicht.

Implementierung

Die Speicherlogik ist in einem benutzerdefinierten React-Hook gekapselt, der das Laden, Speichern und Löschen vollständiger Übersetzungseinträge (einschließlich Bild, Originaltext und Furigana) übernimmt. Dies reduziert die Komplexität der Hauptkomponente und fördert eine modulare, wiederverwendbare Struktur.

React-Hook zur Verwaltung des Übersetzungsverlaufs

Der Hook `useHistory` verwaltet den Zustand des Übersetzungsverlaufs und synchronisiert ihn mit dem persistenten Gerätespeicher. Die Speicherung erfolgt über `AsyncStorage`, ein einfaches Key-Value-System für lokale Daten in React Native. Beim Start der App lädt der Hook vorhandene Einträge und stellt anschließend Funktionen zur Verwaltung bereit:

```
export function useHistory() {
    const [history, setHistory] = useState<Translation[]>([]);
    const loadTranslations = useCallback(async () => {
        try {
            const existingData = await AsyncStorage.getItem(
                ↪ TRANSLATION_STORAGE_KEY);
            if (existingData) {
                setHistory(JSON.parse(existingData) as Translation[]);
            }
        } catch (error) {
            // Fehlerbehandlung
        }
    }, []);

    useEffect(() => {
        loadTranslations();
    }, [loadTranslations]);

    // ...
    return {
        history, saveTranslation, deleteTranslation, clearHistory,
            ↪ loadTranslations,
    };
}
```

Listing 4.23: Auszug aus dem benutzerdefinierten Hook `useHistory`

Ablauf des Speicherprozesses

Der Speicherprozess wird durch den Nutzer manuell über einen Button ausgelöst. Dieser ruft die Funktion `handleSave` auf, die zunächst prüft, ob alle notwendigen Daten (Bild, OCR-Text, Übersetzung) vorhanden sind und ob der Eintrag nicht bereits gespeichert wurde. Anschließend wird das Bild mithilfe von `copyImageToAppStorage` lokal gespeichert und die zugehörigen Daten über `saveTranslation` im Verlauf abgelegt:

```
const handleSave = async () => {
    if (!image || !ocrText || !savedTranslation || isAlreadySaved) return;

    try {
        const savedImageUri = await copyImageToAppStorage(image);
        await saveTranslation(ocrText, savedTranslation, 'en', savedImageUri,
            ↪ furiganaBlocks);
        setIsAlreadySaved(true);
    } catch (error) {
        // Fehlerbehandlung
    }
};
```

Listing 4.24: Einbindung der Speicherfunktion in die Benutzeroberfläche

Einträge hinzufügen und entfernen

Die Funktion `saveTranslation` erstellt einen neuen Eintrag mit Zeitstempel und eindeutiger ID, ergänzt diesen am Anfang der bestehenden Liste und aktualisiert den persistenten Speicher:

```
async function saveTranslation(...) {
    const newEntry = {
        id: Date.now(),
        original,
        translated,
        lang,
        imageUri,
        furigana,
        date: new Date().toISOString(),
    };

    const updatedHistory = [newEntry, ...history];
    await AsyncStorage.setItem(TRANSLATION_STORAGE_KEY, JSON.stringify(
        ↪ updatedHistory));
    setHistory(updatedHistory);
}
```

Listing 4.25: Funktion zur Speicherung eines neuen Eintrags

Einträge lassen sich auch gezielt über ihre ID entfernen. Alternativ kann der gesamte Verlauf gelöscht werden:

```
async function deleteTranslation(id: number) {
    const updatedHistory = history.filter(item => item.id !== id);
    await AsyncStorage.setItem(TRANSLATION_STORAGE_KEY, JSON.stringify(
        ↪ updatedHistory));
    setHistory(updatedHistory);
}

async function clearHistory() {
    await AsyncStorage.removeItem(TRANSLATION_STORAGE_KEY);
    setHistory([]);
}
```

Listing 4.26: Funktionen zum Löschen einzlner Einträge oder des gesamten Verlaufs

4.3.6 Komponente zum PDF-Export

Die App stellt eine Exportfunktion bereit, mit der Beiträge, einschließlich Bild, Originaltext, Furigana und Übersetzung, in ein strukturiertes PDF-Dokument überführt werden können. Dieses kann entweder lokal gespeichert oder über andere Anwendungen geteilt werden. Letzteres erfolgt plattformtypisch über die sogenannte *Freigabe* (engl. *sharing*), also die Übergabe der Datei an kompatible Apps wie E-Mail, Cloudspeicher oder AirDrop. Dieses Verfahren ist unter iOS Standard, da dort kein direkter Zugriff auf das Dateisystem erlaubt ist.
Die Darstellung des Dokuments folgt einem kartenbasierten Layout, in dem die verschiedenen Informationselemente klar gegliedert und optisch ansprechend angeordnet sind. Abbildung 6.1 zeigt ein Beispiel für das generierte PDF.[10]

[10] Yukari (2018, 24. November). *Humble Japanese*. Maggie Sensei. Abgerufen am 4. August 2025, von https://maggiesensei.com/2018/11/24/humble-japanese-%e8%ac%99%e8%ad%b2%e8%aa%9e-kenjougo/



Abbildung 4.7: Beispielhafte Darstellung des erzeugten PDF-Dokuments

Verwendete Technologien

Für die Umsetzung der Komponente kommen mehrere Bibliotheken aus dem Expo-Ökosystem zum Einsatz: `expo-print` wird zur Konvertierung von HTML-Inhalten in PDF-Dateien verwendet. `expo-file-system` ermöglicht den Zugriff auf das Dateisystem sowie die Umwandlung von Dateien in das Base64-Format. `expo-sharing` dient der Übergabe von Dateien an externe Anwendungen.

Implementierung

Die Export-Funktion wird in drei Schritten umgesetzt: Zunächst erfolgt die HTML-basierte Strukturierung der Inhalte, anschließend die Erzeugung einer PDF-Datei und schließlich die plattformabhängige Speicherung oder Freigabe des Dokuments.

Erzeugung der PDF-Datei

Die Funktion `createAndSavePdf` bildet das Herzstück der PDF-Komponente. Sie nimmt ein Objekt mit allen benötigten Inhalten entgegen (OCR-Text, Furigana, Übersetzung, Bild-URI und Datum) und wandelt diese schrittweise in HTML-Fragmente um. Aus diesen Fragmenten wird ein vollständiges HTML-Dokument generiert, das mithilfe von `printToFileAsync` in eine PDF-Datei umgewandelt wird. Die fertige Datei wird anschließend über `saveFile` gespeichert oder zur Freigabe bereitgestellt.

```
1  export async function createAndSavePdf(content: PdfContent) {
2      const { ocrText, furigana, translation, imageUri, date } = content;

4      const imageHtml = imageUri ? await getImageHtml(imageUri) : '';
5      const furiganaHtml = getFuriganaHtml(furigana, ocrText);
6      const translationHtml = translation.replace(/\n/g, '<br />');
7      const dateHtml = date ? formatDateHtml(date) : '';

9      const html = buildPdfHtml({ imageHtml, furiganaHtml, translationHtml,
           ↪ dateHtml });

11     const file = await printToFileAsync({ html, base64: false });
12     await saveFile(file.uri, 'kanjireader.pdf', 'application/pdf');
```

```
13 }
```

Listing 4.27: Funktion zur Erzeugung und Speicherung der PDF-Datei.

Das HTML-Template wird durch die Hilfsfunktion `buildPdfHtml` erzeugt. Es integriert alle zuvor generierten Fragmente in eine klar strukturierte Layoutvorlage, die mithilfe von CSS-Styles formatiert wird:

```
1      ...
2      <body>
3          <div class="card">
4              ${parts.imageHtml}
5              ${parts.furiganaHtml}
6              <hr class="divider" />
7              <div class="textEn">${parts.translationHtml}</div>
8              ${parts.dateHtml}
9          </div>
10     </body>
11     ...
```

Listing 4.28: HTML-Struktur für das PDF.

Speicherung oder Teilen der Datei

Die Funktion `saveFile` übernimmt je nach Plattform die Speicherung oder Freigabe der PDF-Datei.
Unter Android wird zunächst mithilfe des Storage Access Framework (SAF) ein Zielverzeichnis vom Nutzer ausgewählt. Die PDF-Datei wird anschließend im Base64-Format gelesen und in das gewählte Verzeichnis geschrieben. Steht kein Verzeichnis zur Verfügung oder tritt ein Fehler auf, kann die Datei alternativ über das Sharing-Menü geteilt werden.
Auf iOS steht kein direkter Zugriff auf das Dateisystem zur Verfügung, weshalb die Datei grundsätzlich über `shareAsync` an andere Apps übergeben wird.

```
1  async function saveFile(uri: string, filename: string, mimetype: string) {
2      if (Platform.OS === 'android') {
3          const dir = await getOrRequestDirectoryUri();
4          if (!dir) return showSaveFallback(uri);

6          try {
7              const base64 = await FileSystem.readAsStringAsync(uri, {
8                  encoding: 'base64' });
9              const fileUri = await FileSystem.StorageAccessFramework.
                   ↪ createFileAsync(dir, filename, mimetype);
10             await FileSystem.writeAsStringAsync(fileUri, base64, {
11                 encoding: 'base64' });
12         } catch {
13             showSaveFallback(uri);
14         }
15     } else {
16         shareAsync(uri, { UTI: 'public.item' });
17     }
18 }
```

Listing 4.29: Funktion zur asynchronen Speicherung des erzeugten PDF-Dokuments.

5

Qualitätssicherung

Dieses Kapitel widmet sich der Frage, inwieweit die App den Nutzer dabei unterstützt, seine Ziele zu erreichen, wie präzise und zuverlässig die Ergebnisse sind und wie das Gesamterlebnis bei der Nutzung gestaltet ist. Sowohl die technische Leistungsfähigkeit als auch die Benutzerfreundlichkeit stehen dabei im Mittelpunkt.

Kapitel 7.1 widmet sich der Evaluation der drei Hauptfunktionen: OCR, Übersetzung und Furigana-Generierung. Im anschließenden Kapitel 7.2 steht die Untersuchung der Gebrauchstauglichkeit (engl. *Usability*) der App im Fokus.

5.1 Evaluation der Hauptfunktionen

Um die Hauptfunktionen der App zu testen, wurde ein Evaluationsdatensatz mit 73 Beispielen erstellt. Wie in Abbildung 5.1 zu sehen ist, besteht jedes Beispiel aus einem Bild eines japanischen Textes sowie dem dazugehörigen Goldstandard für OCR, Furigana und Übersetzung.

ID	Description	Image	OCR ground truth	Furigana ground truth	Translation ground truth	Source
01	Mix Kanji Katakana Hiragana	image_1.jpg 庭 のバラが咲いている。	庭のバラが咲いている。	庭(にわ)のバラが咲(さ)いている。	The roses in the garden are in flower.	https://jisho.org/search/flower%20%23sentences
02	Mix Kanji Hiragana	image_2.jpg 稲 が実り、田が一面 、黄金色 になっている。	稲が実り、田が一面、黄金色になっている。	稲(いね)が実(みの)り、田(た)が一(いち)面(めん)、黄金色(こがねいろ)になっている。	Rice is ripening and all the rice fields are turning golden.	https://jisho.org/search/all%20%23sentences
03	Hiragana only	image_3.jpg すぐそれをしなければいけませんか。	すぐそれをしなければいけませんか。	-	Do I have to do it right away?	https://tatoeba.org/de/sentences/search?query=%E3
04	Katakana only	image_4.jpg ゴール	ゴール	-	Goal	https://jisho.org/search/%E3%82%B4%E3%83%BC%E

Abbildung 5.1: Ausschnitt aus dem Evaluationsdatensatz. Jeder Eintrag im Datensatz enthält eine ID, eine Beschreibung, ein Bild, die Ground Truths für OCR, Furigana und Übersetzung sowie die Quellenangabe zum Beispielsatz.

Die Beispielsätze stammen unter anderem von den Plattformen Tatoeba, Tofugu, Maggie Sensei oder Jisho. Letzteres greift auf die Datenbank *Jreibun* zu, die von Sprachexperten zusammengestellt wurde und authentische Beispielsätze aus verschiedenen Kontexten enthält. Aufgrund des hohen Aufwands bei der Erstellung eines solchen Datensatzes wurde der Umfang bewusst klein gehalten. Der Fokus

lag insbesondere darauf, Testdaten zu erstellen, die alle relevanten Grenzfälle abdecken. Auf diese Weise sollte ein grober Eindruck der Stärken und Schwächen der App gewonnen werden.
Die Bilder der japanischen Texte wurden im Rahmen der Evaluation durch die App im Android Emulator verarbeitet. Dadurch konnten die OCR-, Furigana- und Übersetzungsfunktionen unter realistischen Bedingungen getestet und die erzielten Ergebnisse systematisch ausgewertet werden, um die Leistungsfähigkeit der App zu beurteilen.

5.1.1 Evaluation der OCR-Funktion

Aufgrund der fehlenden Worttrennungen im Japanischen ist die Zeichenfehlerrate (engl. *Character Error Rate*, CER) besonders gut zur Bewertung von OCR-Ergebnissen geeignet [RRWD22, S. 2]. Sie basiert auf der Levenshtein-Distanz und misst, wie viele Zeichenfehler im Verhältnis zur Länge des Referenztexts auftreten. Die Fehlerquote kann prozentual oder als Dezimalwert zwischen 0 und 1 angegeben werden. Die Formel von CER lautet:

$$\mathrm{CER} = \frac{I + D + S}{N}$$

I ist die Anzahl der Einfügungen (*Insertions*), D ist die Anzahl der Löschungen (*Deletions*), S die Anzahl der Ersetzungen (*Substitutions*) und N die Anzahl der Zeichen im Referenztext. Je niedriger der CER-Wert ist, desto genauer arbeitet das OCR-Modell [NSCT25, S.5]. Obwohl die CER eine etablierte Metrik zur Bewertung von OCR-Systemen ist, existieren bislang keine international standardisierten Referenz-Benchmarks, die CER-Werte für unterschiedliche Anwendungsszenarien vergleichbar machen würden. In der Forschungsliteratur variieren die berichteten CER-Werte stark in Abhängigkeit von eingesetzten Datensätzen, Schriftarten, Sprachen und OCR-Systemen.

Zur automatisierten Evaluierung der mobilen Anwendung kam das Tool *Dinglehopper* zum Einsatz, das im Rahmen des BMBF-geförderten Forschungsprojekts QURATOR entwickelt wurde.[1] [2] Es ermöglicht sowohl die Berechnung einzelner und durchschnittlicher CER-Werte als auch die visuelle Darstellung fehlerhaft erkannter Zeichen durch eine Gegenüberstellung von Ground Truth und OCR-Ergebnis, siehe Abbildung 5.2 [NZB+21, S. 143–144].

[1] Neudecker, C. (2019, 4. Dezember). *Kuratieren mit KI: Erste Ergebnisse aus dem QURATOR-Projekt*. Staatsbibliothek zu Berlin. Abgerufen am 16. Juni 2025, von `https://blog.sbb.berlin/zwischenergebnisse-qurator-jahr-1/`

[2] O. V. (o. D.). *Dinglehopper*. GitHub. Abgerufen am 16. Juni 2025, von `https://github.com/qurator-spk/dinglehopper`



Abbildung 5.2: HTML-basierte Visualisierung des CER-Wertes sowie von OCR-Fehlern mit Dinglehopper. Unterschiede zwischen OCR-Ausgabe und Ground-Truth werden farblich hervorgehoben.

OCR-Ergebnisse gedruckter Texte

Die Qualität der OCR-Erkennung wurde getrennt für handschriftliche und gedruckte Texte bewertet. Die Analyse umfasste insgesamt 56 gedruckte Texte, darunter auch Grenzfälle wie rotierte Layouts, durch Filter überarbeitete Texte sowie Texte mit geringer Auflösung.
Die eingesetzte Google ML Kit API zeigte bei gedruckten Texten insgesamt eine zuverlässige Leistung. Der durchschnittliche CER-Wert (Character Error Rate) lag bei 0,1548, also bei ca. 15 Prozent. Schwächen traten vor allem beim Erkennen von Satzzeichen, einzelnen Kanji-Zeichen und vertikalen Texten auf (siehe Abbildung 5.3).[3]

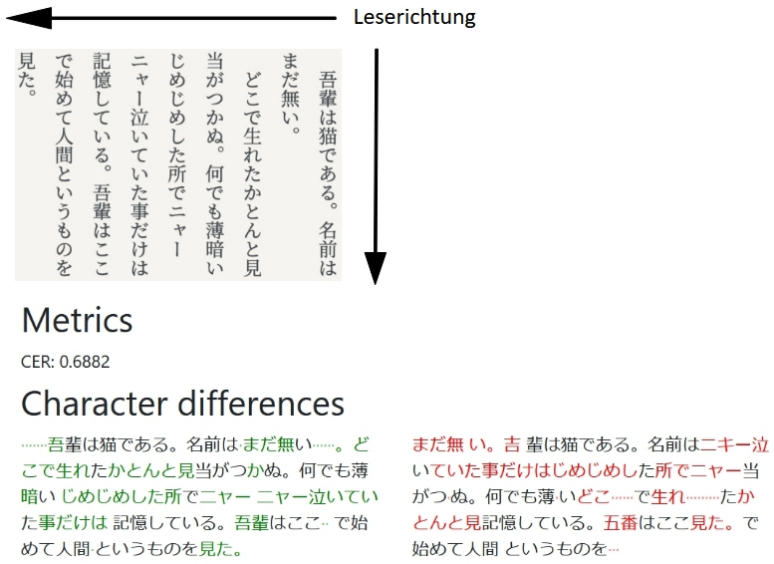
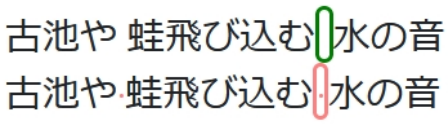


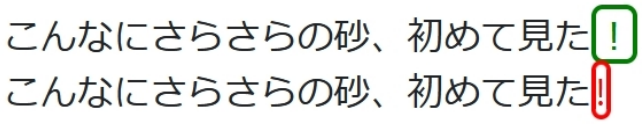
Abbildung 5.3: Beispiel für einen vertikalen Text der zu knapp 70 Prozent fehlerhaft erkannt wurde. Oben ist der Originaltext dargestellt. Unten links ist die Ground Truth zu sehen und unten rechts im Bild das OCR-Ergebnis. Alle Erkennungsfehler wurden farblich hervorgehoben. Zusätzlich ist der CER-Wert für diesen Text angegeben.

[3] O. V. (o. D.). *Bindup*. Abgerufen am 5. Juli 2025, von `https://bindup.jp/camp/bind/24179`

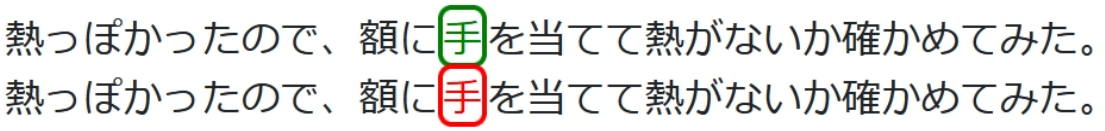
Es ist anzumerken, dass das Tool Dinglehopper sehr streng bewertet. Abbildung 5.4 zeigt exemplarisch, welche Aspekte Dinglehopper unter anderem als Fehler einstuft[4] [5] [6]: Dazu gehören unter anderem das Fehlen von Leerzeichen und die Verwendung verschiedener Unicode-Zeichen für dasselbe Satzzeichen, wie beim Ausrufezeichen im Englischen und Japanischen. Auch Kanji, bei denen anstelle des Zeichens lediglich formverwandte *Radikale* erkannt wurden, werden als Fehler markiert.[7] Dies wirkt sich entsprechend auf den CER-Wert aus.



(a)



(b)



(c)

Abbildung 5.4: Satzelemente, die das Tool Dinglehopper aufgrund seiner genauen Analyse als Fehler einstuft. Der erste Satz zeigt jeweils die Ground Truth, der zweite das OCR-Ergebnis. Fehler sind rot markiert. (a) Fehlende Leerzeichen im Text werden als Fehler erkannt. (b) Ausrufezeichen mit unterschiedlicher Unicode-Codierung gelten als fehlerhaft. (c) Formähnliche Kanji-Zeichen und Radikale mit abweichender Unicode-Codierung werden ebenfalls als Fehler markiert.

OCR-Ergebnisse handschriftlicher Texte

Insgesamt wurden 16 handschriftliche Texte analysiert, davon zwei mit vertikaler Leserichtung, die übrigen in horizontaler Ausrichtung. Die Auswertung zeigt,

[4] O. V. (o. D.). *Haiku*. Wikipedia. Abgerufen am 5. Juli 2025, von `https://en.wikipedia.org/wiki/Haiku`

[5] Dexter, K. (2015, 13. Oktober). *Japanese Onomatopoeia: The Definitive Guide*. Tofugu. Abgerufen am 5. Juli 2025, von `https://www.tofugu.com/japanese/japanese-onomatopoeia/`

[6] O. V. (o. D.). *Jisho.org*. Abgerufen am 5. Juli 2025, von `https://jisho.org/search/hand%23sentences`

[7] Ein Radikal ist ein grafischer Bestandteil eines Kanji, der traditionell zur Kategorisierung und zum Nachschlagen in Wörterbüchern verwendet wird. Quelle: O. V. (o. D.). *Chinese character radicals*. Wikipedia. Abgerufen am 7. Juli 2025, von `https://en.wikipedia.org/wiki/Chinese_character_radicals`

dass das OCR-Modell bei handschriftlichen Inhalten eine deutlich höhere Fehlerquote aufweist als bei gedrucktem Text. Besonders kalligrafisch gestaltete sowie vertikal ausgerichtete Texte bereiteten dem Modell erhebliche Schwierigkeiten. Der durchschnittliche CER-Wert lag im verwendeten Datensatz bei 0,3384, was einer Fehlerquote von etwa 34 Prozent entspricht.

5.1.2 Evaluation der Furigana-Funktion

Kuromoji wird in vielen praktischen Anwendungen erfolgreich eingesetzt und gilt als zuverlässig. Die Evaluation konzentrierte sich daher nicht auf eine umfassende quantitative Bewertung der Lesungszuordnung, sondern gezielt auf Randfälle, in denen Kuromoji fehlerhafte oder unvollständige Furigana generierte. Zu diesem Zweck wurde eine qualitative Fehleranalyse durchgeführt, um typische Probleme systematisch zu identifizieren und zu klassifizieren. Die resultierenden Fehlertypen lieferten Hinweise auf potenzielle Schwächen des zugrunde liegenden Wörterbuchs und können als Grundlage für eine gezielte Optimierung dienen.

Zur systematischen Erfassung der beobachteten Fehler wurden zunächst drei Hauptfehlerklassen definiert (siehe Tabelle 5.1). Diese dienten bei der ersten Sichtung der Daten als Orientierungshilfe, insbesondere zur Unterscheidung zwischen Fehlern, die auf unzureichende OCR-Ergebnisse zurückgingen, und solchen, die durch eine fehlerhafte Analyse seitens Kuromoji verursacht wurden. Für Fehler der Klasse 1 (Falsche Lesung) wurden zusätzlich Unterkategorien eingeführt, um die Ursachen differenzierter erfassen zu können (siehe Tabelle 5.2).

Tabelle 5.1: Hauptfehlerklassen bei der Furigana-Generierung

ID	Fehlerklasse	Beschreibung
1	Falsche Lesung	Das Furigana ist formal korrekt, entspricht aber nicht der erwarteten Lesung des Tokens.
2	Fehlendes Furigana	Für ein Kanji wurde keine Lesung generiert (Ausgabe als ∗), obwohl eine Lesung erwartet wurde.
3	OCR-Artefakt	Das fehlerhafte Furigana ist auf eine falsche Zeichenerkennung im OCR-Prozess zurückzuführen.

Tabelle 5.2: Potentielle Fehlerursachen für Klasse 1 (Falsche Lesung)

ID	Fehlerursache	Beschreibung
1.1	Mehrere Lesungen	Das Token besitzt mehrere zulässige Lesungen (z. B. Kun- und On-Lesung); Kuromoji wählt jedoch eine kontextuell unpassende Variante, etwa die lexikalisch dominantere.
1.2	Falsche Wortsegmentierung	Kuromoji erkennt ein zusammengesetztes Wort (Kompositum) nicht und segmentiert es fälschlich in mehrere Einzeltoken.
1.3	Fehlende oder unvollständige Lexikoneinträge	Die Wortform ist im zugrunde liegenden Wörterbuch nicht enthalten oder wird mit einer unüblichen Lesung verknüpft.

Die Zuordnung zu diesen Unterkategorien erfolgte auf Basis der beobachtbaren Ausgabe und typischer lexikalischer Muster. Allerdings ließ sich die interne Verarbeitung durch Kuromoji nicht in allen Fällen eindeutig nachvollziehen. In Einzelfällen war daher unklar, ob ein Fehler etwa auf eine fehlerhafte Segmentierung oder unvollständige Lexikoneinträge zurückzuführen war. Die Klassifikation ist somit als plausible, aber nicht in jedem Fall eindeutig überprüfbare Ursachenanalyse zu verstehen.

Ein exemplarischer Ausschnitt aus der Fehlerdokumentation ist in Abbildung 5.5 dargestellt. Er veranschaulicht das verwendete tabellarische Format sowie die Art der protokollierten Informationen.

ID	Originaltext (Kanji)	Ziel-Furigana	System-Furigana	Fehlerklasse	Kommentar / Ursache
2	黄金色	こがねいろ	おうごんしょく	Falsche Lesung	Beide Lesungen sind korrekt, aber „おうごんしょく“ ist die Standard-On-Lesung, „こがねいろ“ hingegen poetischer und seltener. → Mehrdeutige Lesung
9	触	ふ	どく	OCR-Artefakt	OCR-Ausgabe: 独
9	幼	おさな	-	OCR-Artefakt	OCR-Ausgabe: h
10	餌	えさ	お	OCR-Artefakt	OCR-Ausgabe: 御
10	鳩	はと	ち	OCR-Artefakt	OCR-Ausgabe: 地
10	褒美	ほうび	* ぎょう	OCR-Artefakt, Fehlendes Furigana	OCR-Ausgabe: 衰業

Abbildung 5.5: Ausschnitt aus der tabellarischen Dokumentation der Furigana-Fehler. Erfasst sind jeweils die ID des Beispiels, der Originaltext (Kanji), das erwartete und das tatsächlich generierte Furigana, die zugewiesene Fehlerklasse sowie ein Kommentar zur vermuteten Ursache.

Evaluationsergebnisse

Im Testdatensatz wurden insgesamt 47 Fehler bei der Furigana-Generierung dokumentiert. Rund zwei Drittel davon (31 Fälle) ließen sich auf fehlerhafte OCR-

Ergebnisse zurückführen (Fehlerklasse 3). Häufig führten Verwechslungen mit formähnlichen Kanji oder Katakana dazu, dass Token keine oder unpassende Lesungen erhielten. In einigen Fällen wurden Furigana an falschen Stellen eingefügt oder fehlten vollständig. Solche Fehler waren ausschließlich auf mangelhafte Zeichenerkennung zurückzuführen, nicht auf die Analysekomponenten des Systems. Weitere Probleme entstanden durch fälschlich erkannte Leerzeichen zwischen Kanji, die die Tokenisierung und damit die Lesungszuordnung beeinträchtigten.
Daneben wurden 16 Fehler der Klasse 1 (Falsche Lesung) erfasst und den zuvor definierten Subkategorien zugeordnet. Die Mehrheit (11 Fälle) betraf mehrdeutige Lesungen (Kategorie 1.1), bei denen Kuromoji aus mehreren möglichen Varianten (z. B. On- oder Kun-Lesung) eine zwar formal korrekte, aber kontextuell unpassende auswählte.
Die übrigen fünf Fälle entfielen auf die Kategorien 1.2 (Falsche Wortsegmentierung) und 1.3 (Fehlende oder unvollständige Lexikoneinträge). Entweder wurden Komposita fälschlich aufgespalten, was zu fehlerhaften Lesungen führte, oder bestimmte Wortformen bzw. Lesungsvarianten waren im Lexikon nicht oder nur unzureichend erfasst. Segmentierungsfehler beeinträchtigten dabei oft auch die nachfolgende Morphemanalyse.

Insgesamt zeigt die Analyse, dass die zentralen Herausforderungen bei der automatischen Furigana-Generierung weniger auf technische Defizite zurückgehen, sondern primär in der inhärenten Ambiguität der japanischen Schriftsprache liegen. Eine zuverlässige Lesungszuordnung erfordert daher kontextsensitive Verfahren sowie eine sorgfältige Pflege des Lexikons und eine robuste Segmentierungsstrategie.

5.1.3 Evaluation der Übersetzungsfunktion

Methoden zur Bewertung maschineller Übersetzungen lassen sich grundsätzlich in zwei Kategorien einteilen: manuelle und automatische Verfahren. Manuelle Ansätze basieren auf qualitativen Urteilen durch menschliche Gutachter und gelten als besonders zuverlässig. Sie sind jedoch zeit- und ressourcenintensiv und liefern aufgrund ihrer Subjektivität oft schwer reproduzierbare Ergebnisse [LLM+23, S. 1–2].
Automatische Metriken bieten eine zeitsparende und kostengünstige Alternative und werden insbesondere bei der Entwicklung moderner Übersetzungssysteme eingesetzt. Dabei erfolgt die Bewertung in der Regel durch den Vergleich der Systemausgabe mit einer oder mehreren Referenzübersetzungen. Diese Verfahren erfassen jedoch nicht die tatsächliche Übersetzungsqualität, sondern lediglich die Übereinstimmung mit den Referenztexten [RT22, S. 20].
Um diese methodischen Einschränkungen zu verringern, wurden in den letzten Jahren Bewertungsverfahren entwickelt, die auch syntaktische und semantische Aspekte berücksichtigen. Viele dieser neueren Metriken basieren auf Deep-Learning-Techniken und orientieren sich stärker an menschlichen Beurteilungskriterien, um die Aussagekraft automatisierter Bewertungen zu erhöhen [Han16, S. 4–5].

Ein zentrales Problem bleibt jedoch bestehen: Anders als etwa in der Mathematik gibt es in der Sprachübersetzung keine eindeutig richtige Lösung. Ein und derselbe Ausgangstext kann auf verschiedene Weise korrekt wiedergegeben werden. Diese inhärente Mehrdeutigkeit stellt für manuelle wie automatische Verfahren eine bleibende Herausforderung dar [RT22, S. 20].

Automatisierte Evaluation mit MATEO

Zur automatisierten Bewertung der Übersetzungen kam im Rahmen dieses Projekts die Webanwendung MATEO (*MAchine Translation Evaluation Online*) zum Einsatz [VTM23]. Dabei handelt es sich um ein Projekt der Universität Gent mit dem Ziel, automatisierte Metriken einer breiten Öffentlichkeit zugänglich zu machen. MATEO kann lokal installiert werden und unterstützt derzeit sechs Metriken: BLEU, ChrF, TER, BLEURT, BERTScore und COMET.

Als Eingabe erwartet MATEO mindestens drei UTF-8-codierte Textdateien: eine mit den japanischen Ausgangstexten, eine mit den Referenzübersetzungen ins Englische (Goldstandard) sowie eine mit den Hypothesen, also den Systemausgaben des zu bewertenden Modells. Optional lassen sich bis zu drei weitere Hypothesendateien hinzufügen, um mehrere Systeme miteinander zu vergleichen. Jede Zeile repräsentiert dabei genau einen Datenpunkt.

Für eine differenzierte Einschätzung der Übersetzungsqualität wurden zwei komplementäre Metriken ausgewählt: eine klassische, n-gram-basierte Methode (BLEU) und eine moderne, auf neuronalen Netzwerken basierende Metrik (COMET). Beide unterscheiden sich deutlich in Methodik, Rechenaufwand und Aussagekraft:

- **BLEU** (*Bilingual Evaluation Understudy*) ist eine der ältesten und am weitesten verbreiteten automatisierten Metriken zur Evaluierung maschineller Übersetzungen.[8] Die Grundidee besteht darin, die n-Gram-Überlappung zwischen der Systemausgabe (Hypothese) und einer oder mehreren Referenzübersetzungen zu messen [PRWZ01].[9]
  Zur Berechnung wird die sogenannte *modifizierte n-Gram-Präzision* verwendet, bei der jedes n-Gramm in der Hypothese nur so oft gezählt wird, wie es in den Referenzübersetzungen tatsächlich vorkommt.[10] Um extrem kurze Hypothesen zu vermeiden, wird zusätzlich ein Strafterm eingeführt: die sogenannte *Brevity Penalty* (BP). Der endgültige BLEU-Wert ergibt sich als geometrisches Mittel der modifizierten Präzisionen für $n = 1$ bis N, multipliziert mit der Brevity Penalty. Der Score liegt im Intervall $[0, 1]$, wobei höhere Werte auf eine größere

[8] Vashee, K. (2021, 25. Oktober). *Understanding MT Quality: BLEU Scores*. modernMT. Abgerufen am 26. Juni 2025, von `https://blog.modernmt.com/understanding-mt-quality-bleu-scores/`

[9] Ein n-Gramm bezeichnet eine Folge von n aufeinanderfolgenden Wörtern in einem Text.

[10] Die Präzision (engl. *precision*) bezeichnet den Anteil der n-Gramme in der Hypothese, die auch in mindestens einer Referenzübersetzung enthalten sind. Sie berechnet sich als Quotient aus der Anzahl übereinstimmender n-Gramme und der Gesamtzahl der n-Gramme in der Hypothese.

formale Übereinstimmung mit den Referenzen hinweisen. Die Berechnungsformel lautet:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \cdot \log p_n\right) \tag{5.1}$$

mit:

p_n	Modifizierte n-Gram-Präzision
w_n	Gewicht des jeweiligen n-Grams, meist: $w_n = \frac{1}{N}$
N	Maximale n-Gram-Länge, typischerweise $N = 4$
BP	Strafterm für zu kurze Hypothesensätze (Brevity Penalty)

Die Brevity Penalty berechnet sich wie folgt:

$$\text{BP} = \begin{cases} 1, & \text{wenn } c > r \\ e^{(1-\frac{r}{c})}, & \text{wenn } c \leq r \end{cases} \tag{5.2}$$

wobei:

c	Gesamtlänge der Hypothesentexte
r	Effektive Länge der Referenztexte

BLEU erlaubt eine effiziente und reproduzierbare Bewertung auf Korpusebene und eignet sich daher besonders für den systematischen Vergleich von Übersetzungsmodellen. Aufgrund seiner rein formalen Orientierung erkennt BLEU jedoch keine semantischen Äquivalenzen und reagiert empfindlich auf Variationen in Wortwahl oder Satzstruktur. Eine korrekte, aber anders formulierte Übersetzung kann somit einen niedrigen BLEU-Wert erhalten.

- **COMET** (*Crosslingual Optimized Metric for Evaluation of Translation*) ist ein modernes Framework zur automatisierten Bewertung maschineller Übersetzungen.[11] Es berechnet die semantische Ähnlichkeit zwischen einer maschinellen Übersetzung und einer Referenzübersetzung auf Basis von Token- oder Satz-*Embeddings*.[12] Im Unterschied zu vielen anderen automatisierten Metriken berücksichtigt COMET auch den Quelltext und ist darauf trainiert, menschliche Qualitätsurteile möglichst präzise vorherzusagen [RSFL20].
  Technisch basiert COMET auf einem vortrainierten, mehrsprachigen Sprachmodell (typischerweise XLM-RoBERTa), das durch Fine-Tuning an die Aufgabe angepasst wird. Trainiert wird es mit verschiedenen Formen menschlicher Bewertungen, etwa Direct Assessments (DA), Human-targeted Translation Edit Rate (HTER) oder Fehlerannotationen.
  Die Eingabetexte (Quelltext, Hypothese und Referenz) werden im Modell zunächst in semantische Embeddings überführt und anschließend gemeinsam

[11] O. V. (o. D.). *Unbabel/COMET*. GitHub. Abgerufen am 04.08.2025, von `https://github.com/Unbabel/COMET`

[12] Ein Embedding ist eine dichte, numerische Repräsentation eines sprachlichen Elements (z. B. eines Satzes) in einem kontinuierlichen Vektorraum. Embeddings ermöglichen es neuronalen Netzen, semantische Ähnlichkeiten zwischen sprachlichen Einheiten rechnerisch zu erfassen.

analysiert. Je nach Einsatzzweck kommen zwei unterschiedliche Modelltypen zum Einsatz (vgl. Abbildung 5.6):

– Der *Estimator* gibt einen kontinuierlichen Qualitätswert aus, der mithilfe eines Regressionsverfahrens bestimmt wird. Dabei versucht das Modell, sich numerischen Zielwerten möglichst genau anzunähern.

– Das *Ranking-Modell* lernt, im Vergleich zweier Übersetzungen die bessere zuverlässig zu identifizieren. Grundlage dafür ist der sogenannte Triplet Loss, ein Lernverfahren, das darauf ausgelegt ist, relative Qualitätsunterschiede zu erkennen.

COMET erkennt semantische Ähnlichkeiten auch bei abweichender Formulierung und erreicht dadurch eine deutlich bessere Übereinstimmung mit menschlichen Bewertungen als klassische Metriken wie BLEU. Der Einsatz setzt jedoch ein vortrainiertes Modell voraus und ist mit einem höheren Rechenaufwand verbunden.

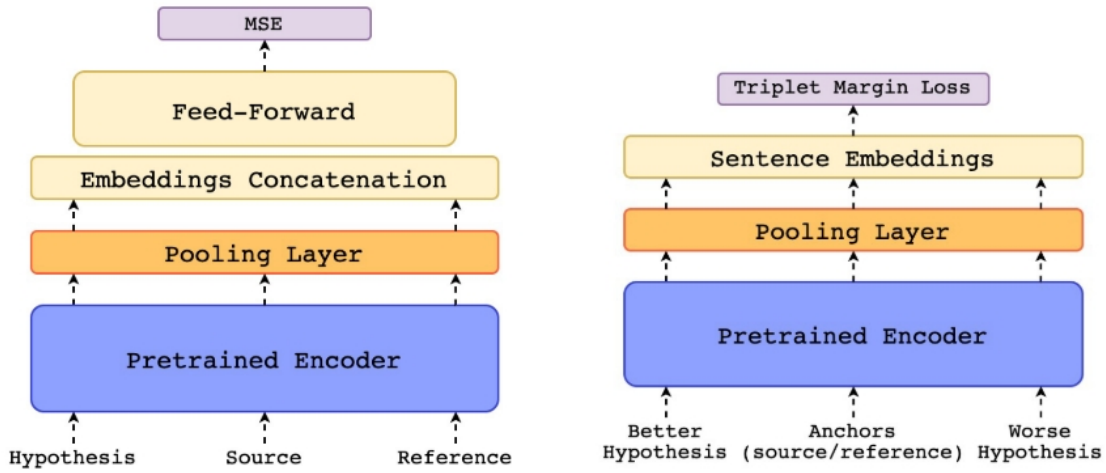


Abbildung 5.6: *Links*: Architektur des Estimator-Modells. Quelltext, Hypothese und Referenz werden separat durch ein vortrainiertes Sprachmodell kodiert und zu Satz-Embeddings verdichtet. Diese werden kombiniert, konkateniert und anschließend von einem Regressor bewertet. Das Modell wird durch Minimierung des mittleren quadratischen Fehlers (MSE) trainiert. *Rechts*: Architektur des Ranking-Modells. Quelltext, Referenz sowie eine bessere und eine schlechtere Hypothese werden einzeln kodiert und zu Satz-Embeddings zusammengefasst. Die Embeddings werden so optimiert, dass die bessere Hypothese im Vektorraum näher an Quelle und Referenz liegt als die schlechtere (Training mittels Triplet-Loss) [RSFL20].

MATEO erlaubt die Anpassung verschiedener Parameter für die Berechnung der ausgewählten Metriken. Im Rahmen dieses Projekts wurden sowohl für BLEU als auch für COMET die Standardeinstellungen verwendet (vgl. Abbildung 5.7). Für COMET kam dabei das Modell `Unbabel/wmt22-comet-da` zum Einsatz, das auf *Direct Assessment* (DA) trainiert wurde und Qualitätswerte im Bereich von 0 bis 1 ausgibt, wobei ein Wert von 1 einer perfekten Übersetzung entspricht.[13] Die BLEU-Werte wurden mithilfe der standardisierten SacreBLEU-Implementierung berechnet.[14]

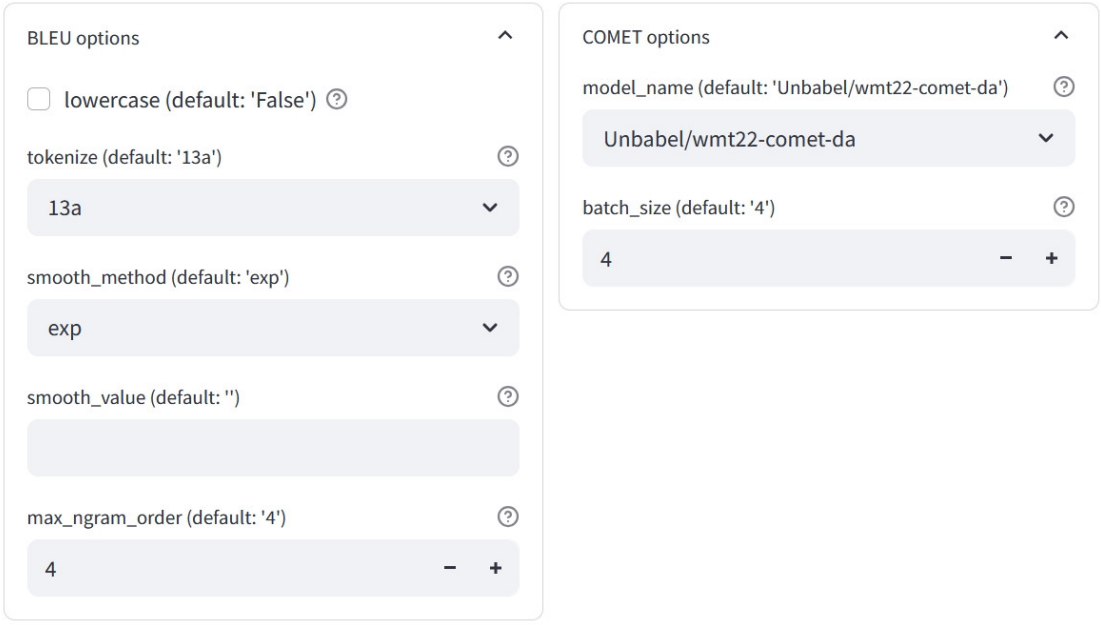


Abbildung 5.7: Metrikkonfiguration in MATEO.

Datenbereinigung und Vergleichsgrundlage

Um Verzerrungen bei der Bewertung zu minimieren, wurde der Evaluationsdatensatz vorab bereinigt. Dafür wurden alle Duplikate entfernt sowie Instanzen ausgeschlossen, bei denen keine OCR-Ergebnisse vorlagen, etwa Bilder ohne erkennbaren Text. Dadurch verringerte sich die Anzahl der Beispieltexte von ursprünglich 73 auf 59.
Anschließend wurde der bereinigte Datensatz genutzt, um Vergleichswerte auf Basis der originalen, OCR-freien Referenztexte zu erzeugen. Hierfür wurde ein JavaScript-Programm entwickelt, das die Übersetzungslogik der App simuliert und Anfragen an das Übersetzungsmodell sendet. Dies ermöglichte eine gezielte Analyse der Auswirkungen von OCR-bedingten Fehlern auf die Übersetzungsqualität.

[13] Direct Assessment (DA) bezeichnet eine menschliche Bewertungsmethode, bei der Übersetzungen direkt auf einer Skala (z. B. 0 bis 100) hinsichtlich ihrer Qualität bewertet werden.
[14] O. V. (o. D.). *mjpost/sacrebleu*. GitHub. Abgerufen am 04.08.2025, von `https://github.com/mjpost/sacrebleu`

Evaluationsergebnisse

Tabelle 5.3 zeigt die Mittelwerte der Evaluationsmetriken BLEU und COMET für das verwendete Übersetzungsmodell `Helsinki-NLP/opus-mt-ja-en`. Verglichen werden die Ergebnisse bei OCR-basierten Eingabedaten (*ocr_based*) mit denen auf Basis der ursprünglichen, fehlerfreien Referenztexte (*reference_based*). Die OCR-basierte Variante dient dabei als Referenzwert (*Baseline*).
Beide Metriken zeigen einen deutlichen Qualitätsunterschied: Die Nutzung von OCR-basierten Eingaben führt zu einem relativen Qualitätsverlust von etwa 12,3 % laut COMET und rund 26 % laut BLEU. Diese Werte verdeutlichen, dass Fehler in der Texterkennung die Übersetzungsleistung der App erheblich beeinträchtigen (vgl. Abbildung 5.8).

Tabelle 5.3: Evaluationsergebnisse berechnet mit MATEO. Die Tabelle zeigt sowohl Korpus-Ergebnisse als auch Mittelwerte (μ) mit 95% Konfidenzintervallen in Klammern. Die p-Werte geben die statistische Signifikanz im Vergleich zur Baseline (*ocr_based*) an. Ein Sternchen (*) markiert Ergebnisse, die signifikant besser sind als die Baseline ($p < 0{,}05$).

System	**COMET** ($\mu \pm 95\%$ CI)	**BLEU** ($\mu \pm 95\%$ CI)
ocr_based	69.4 (69.4 ± 4.8)	16.5 (16.3 ± 4.9)
reference_based	79.1 (79.1 ± 3.3) (p = 0.0010)*	22.3 (22.5 ± 5.7) (p = 0.0270)*

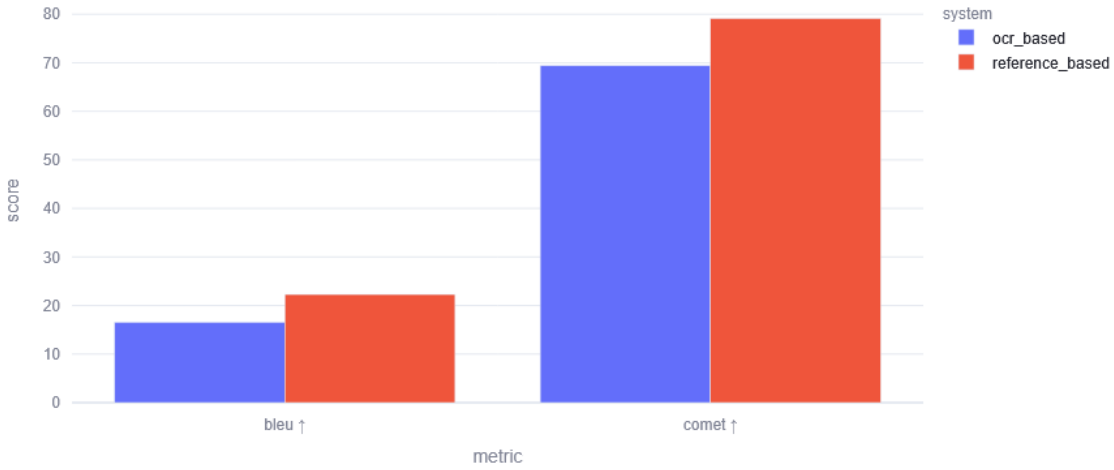


Abbildung 5.8: Vergleich der BLEU- und COMET-Werte auf Systemebene.

Die BLEU-Werte beider Varianten liegen unterhalb der allgemein anerkannten Verständlichkeitsschwelle von etwa 30 Punkten.[15] Die Baseline (*ocr_based*) erreicht lediglich 16,55 Punkte und befindet sich damit in einem Bereich, der typischerweise mit schwer verständlichen Übersetzungen assoziiert wird. Die *reference_based*-Variante erzielt 22,28 Punkte und liegt damit am unteren Rand des Bereichs, in dem die wesentlichen Inhalte noch erkennbar sind, allerdings weiterhin mit deutlichen sprachlichen Mängeln (vgl. Abbildung 5.9).

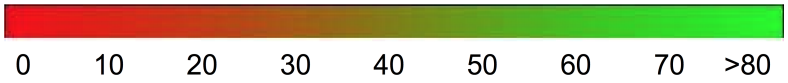


Abbildung 5.9: Farbverlauf als generelle Skala zur Interpretation des BLEU-Scores.[15]

Im Gegensatz dazu zeichnen die deutlich höheren COMET-Werte ein insgesamt positiveres Bild. Sie legen nahe, dass das Modell grundsätzlich in der Lage ist, inhaltlich zutreffende und verständliche Übersetzungen zu erzeugen, insbesondere bei qualitativ hochwertigen Eingabedaten. Selbst die OCR-basierte Variante erreicht mit einem Score von 69,4 einen vergleichsweise hohen Wert, was auf eine insgesamt robuste Modellleistung hindeutet.

Die Diskrepanz zwischen beiden Metriken legt nahe, dass sich die erzeugten Übersetzungen zwar deutlich in Form und Struktur von den Referenzen unterscheiden, den Inhalt jedoch im Wesentlichen korrekt wiedergeben. COMET kann solche semantischen Übereinstimmungen auch bei abweichender Wortwahl erfassen und liefert dadurch ein differenzierteres Bild der tatsächlichen Übersetzungsqualität.
Abbildung 5.10 zeigt dies exemplarisch an einem Beispielsatz aus dem Testdatensatz: Trotz deutlicher formaler Abweichungen gibt die Systemübersetzung die Bedeutung korrekt wieder und erreicht einen COMET-Score von etwa 91. Metriken wie BLEU erkennen solche semantischen Übereinstimmungen oft nicht und bewerten entsprechend niedriger.

[15] Lavie, A. (2011, 19. September). *Evaluating the Output of Machine Translation Systems.* Abgerufen am 5. Juli 2025, von `https://cloud.google.com/translate/docs/advanced/automl-evaluate`

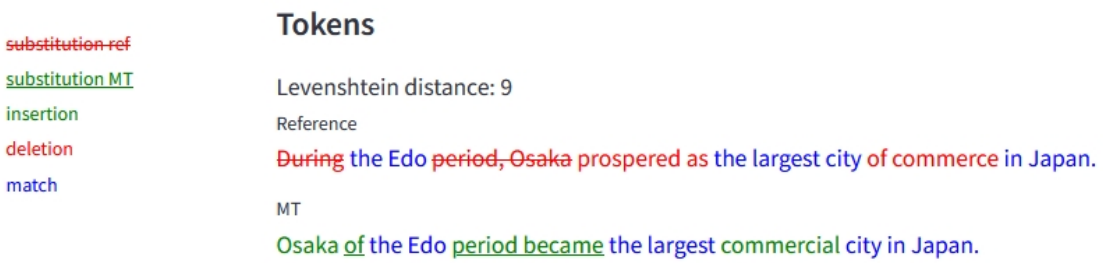


Abbildung 5.10: Visualisierung der Unterschiede zwischen Hypothese und Referenz. Die mit MATEO erzeugte Darstellung zeigt Bearbeitungsoperationen zur Angleichung der Systemausgabe an die Referenz. Trotz formaler Abweichungen erkennt COMET eine hohe semantische Übereinstimmung (Score ≈ 91).

Über das Beispiel hinaus bieten die COMET-Scores auf Satzebene Hinweise darauf, mit welchen Arten von Eingaben das Modell besonders gut oder weniger gut zurechtkommt. Dabei ist jedoch zu beachten, dass der verwendete Testdatensatz vergleichsweise klein ist und nur eingeschränkt Rückschlüsse auf die spezifischen Stärken und Schwächen des Modells zulässt.
Niedrige Bewertungen (unter 70 Punkte) traten vor allem bei Redewendungen und literarischen Texten auf, darunter zwei Haiku-Gedichte. Auch Eingaben mit ungewöhnlicher Struktur, etwa Liedtexte oder Zutatenlisten, führten häufiger zu fehlerhaften Übersetzungen. Mehrere schwach bewertete Ausgaben basierten zudem auf Eingaben mit japanischer Onomatopoesie.[16]
Hohe COMET-Scores erzielte das Modell vor allem bei kurzen, strukturell einfachen Eingaben mit geringer bis mittlerer Komplexität. Auf Basis dieser Beobachtung wurde die Übersetzungsfunktion um einen Vorverarbeitungsschritt erweitert: Längere Texte werden nun in Abschnitte von jeweils zwei Sätzen unterteilt, gemeinsam übersetzt und anschließend zu einem Gesamtergebnis zusammengeführt. Tests zeigten, dass sich dadurch der Informationsverlust verringern und die Übersetzungsqualität insgesamt verbessern ließ.

5.2 Usability Testing

Die *Gebrauchstauglichkeit* (engl. *Usability*) eines Produkts bezeichnet gemäß DIN EN ISO 9241-11 das Maß, in dem bestimmte Nutzer ein Produkt in einem definierten Nutzungskontext verwenden können, um festgelegte Ziele effektiv, effizient und zufriedenstellend zu erreichen.[17] [18] Im Rahmen des Teamprojekts wurde die

[16] Japanische Onomatopoesie bezeichnet lautmalerische Ausdrücke, die nicht nur Geräusche, sondern auch Bewegungen, Zustände oder Gefühle nachahmen. Sie ist ein fester Bestandteil der Alltagssprache und findet sich häufig in informellen Texten, Manga und Werbung.

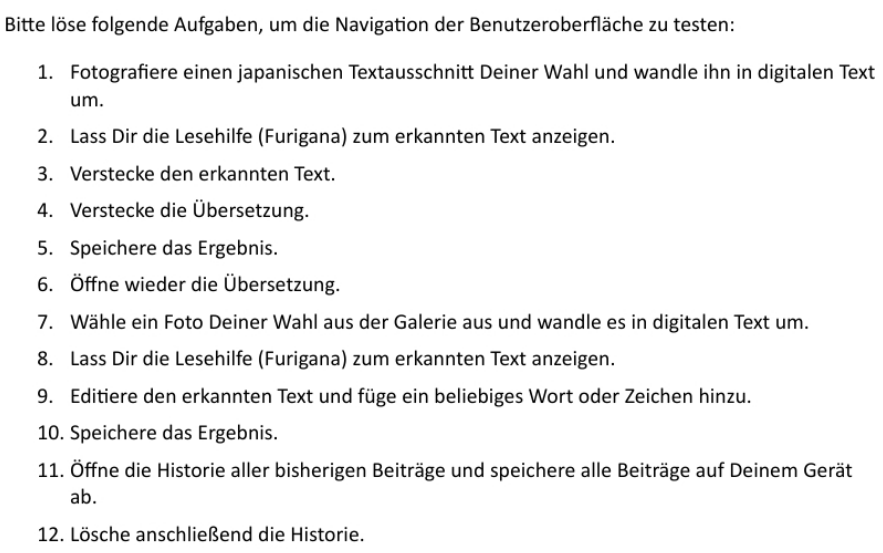
[17] Der Nutzungskontext umfasst nach ISO 9241-11 alle Bedingungen, unter denen ein Produkt von Personen genutzt wird. Dazu zählen Nutzer, Ziele, Aufgaben, Ressourcen sowie die technische, physische, soziale, kulturelle und organisatorische Umgebung. Quelle: O. V. (o. D.). *Nutzungskontext*. Wikipedia. Abgerufen am 7. Juli 2025, von `https://de.wikipedia.org/wiki/Nutzungskontext`

[18] O. V. (o. D.). *Gebrauchstauglichkeit (Produkt)*. Wikipedia. Abgerufen am 26. Mai 2025, von `https://de.wikipedia.org/wiki/Gebrauchstauglichkeit_(Produkt)`

Usability der App mit sieben Personen getestet, die Japanisch im Selbststudium oder im Rahmen eines formalen Studium lernen. Gemäß dem dänischen Usability-Experten Jakob Nielsen reicht in einer qualitativen Usability-Studie bereits eine kleine Testgruppe aus, um 85 Prozent der häufigsten Usability-Probleme zu identifizieren:

> „Aufwändige Usability-Tests sind eine Verschwendung von Ressourcen. Die besten Ergebnisse erzielt man, wenn man nicht mehr als fünf Nutzer testet und so viele kleine Tests wie möglich durchführt." –*Jakob Nielsen*[19]

Ziel des Testverfahrens war es demzufolge, grundlegende Schwächen der App zu identifizieren, um diese in einer späteren Entwicklungsphase gezielt zu verbessern. Die Teilnehmenden bearbeiteten Aufgaben auf einem Android-Gerät (vgl. Abbildung 5.11) und beantworteten drei Zusatzfragen. Außerdem füllten die Testpersonen den von Schrepp et al. entwickelten *User Experience Questionnaire* (UEQ) aus.[20] Dabei handelt es sich um einen standardisierten Fragebogen, mit dessen Hilfe systematisch die Nutzererfahrung eines interaktiven Produkts ermittelt werden kann. Da der UEQ jedoch eine quantitative Messmethode darstellt, die ihren Entwicklern zufolge erst ab 20 bis 30 Teilnehmenden belastbare statistische Ergebnisse liefert [Sch23, S.15], wurden die Ergebnisse des Fragebogens aufgrund der mangelnden Aussagekraft infolge der zu geringen Teilnehmerzahl verworfen.


Bitte löse folgende Aufgaben, um die Navigation der Benutzeroberfläche zu testen:

1. Fotografiere einen japanischen Textausschnitt Deiner Wahl und wandle ihn in digitalen Text um.
2. Lass Dir die Lesehilfe (Furigana) zum erkannten Text anzeigen.
3. Verstecke den erkannten Text.
4. Verstecke die Übersetzung.
5. Speichere das Ergebnis.
6. Öffne wieder die Übersetzung.
7. Wähle ein Foto Deiner Wahl aus der Galerie aus und wandle es in digitalen Text um.
8. Lass Dir die Lesehilfe (Furigana) zum erkannten Text anzeigen.
9. Editiere den erkannten Text und füge ein beliebiges Wort oder Zeichen hinzu.
10. Speichere das Ergebnis.
11. Öffne die Historie aller bisherigen Beiträge und speichere alle Beiträge auf Deinem Gerät ab.
12. Lösche anschließend die Historie.

Abbildung 5.11: Test-Szenarien, die von den Probanden durchlaufen wurden.

[19] Nielsen, J. (2000, 18. März). *Why You Only Need to Test with 5 Users.* Nielsen Norman Group. Abgerufen am 7. Juli 2025, von `https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/`

[20] O. V. (o. D.). *Offizielle Website zum UEQ-Tool.* UEQ Online. Abgerufen am 30. Juni 2025, von `https://www.ueq-online.org/`

5.2.1 Testergebnisse

Das Testverfahren zeigte, dass die Teilnehmenden unterschiedlich gut mit der UI-Navigation zurechtkamen. Während einige die App als „intuitiv “und „einfach zu verstehen “beschrieben, empfanden andere die Abläufe als „etwas verwirrend “und betonten, dass eine gewisse Übung erforderlich sei, um die einzelnen Schritte routiniert durchführen zu können.

Folgende Aspekte der App wurden im Zuge des Testverfahrens kritisiert: So war die Cropping-Funktion für zwei Teilnehmende nicht selbsterklärend. Auch der Zweck der Editierfunktion sowie der „Hide “-Buttons blieb für einige unklar. Eine Testperson schlug vor, statt einer Exportfunktion eine Kopiermöglichkeit für den japanischen Text und dessen Übersetzung anzubieten. Darüber hinaus wurde der Wunsch nach einer kleineren Textdarstellung zur besseren Übersicht geäußert. Eine weitere Rückmeldung bezog sich auf die grundsätzliche Funktionalität der App: Zwar demonstriere sie überzeugend, wie sich zentrale Funktionen realisieren ließen, erreiche jedoch noch keinen „Werkzeug-Status “, da die zugrunde liegenden sprachtechnologischen Komponenten derzeit noch fehlerhafte Ergebnisse lieferten.

6

Ergebnisse

In diesem Kapitel wird der Entwicklungsstand der App zum Projektende beschrieben. Zunächst wird die im UI-Design eingesetzte Farbpalette vorgestellt. Anschließend veranschaulichen ausgewählte Screenshots die zentralen Funktionen der finalen App-Version im praktischen Einsatz. Den Abschluss bildet eine Übersicht technischer Einschränkungen, Herausforderungen sowie noch offener Aufgabenbereiche.

6.1 Farbpalette

Die App verwendet eine ausgewogene Farbpalette aus warmem Dunkelrot und neutralen Beigetönen. Ein tiefes Weinrot (#752C2C) prägt Header und Footer, während ein sehr helles Cremebeige (#FBF7F0) als Hintergrund dient. Schwarze Schrift (#000000) sorgt dabei für gute Lesbarkeit.
Buttons sind in einem satten Rotbraun (#863333) gehalten, das sich bei Interaktion zu einem helleren Rotton (#A73F3F) verändert. Dezente Grautöne (#898989) und ein weiches Gelbbeige (#F5F0CC) bilden die Rahmenfarben. Die Beschriftung erfolgt in Weiß (#FFFFFF) bzw. im aktiven Zustand ebenfalls in Gelbbeige (#F5F0CC), um Kontrast und visuelles Feedback zu sichern. Alle Farbkombinationen erfüllen mindestens die WCAG-Kontrastanforderungen auf AA-Niveau und unterstützen eine barrierefreie Nutzung.



Abbildung 6.1: Farbpalette der App

6.2 Abschließende App-Version

Die finale Version der Anwendung ist für Android-Geräte konzipiert und verwendet optische Zeichenerkennung (OCR), um japanische Texte auf Fotos zu identifizieren.

Der erkannte Text wird automatisch ins Englische übersetzt, und Kanji können auf Wunsch zusätzlich mit Furigana versehen werden. Darüber hinaus ermöglicht die App die nachträgliche Bearbeitung des erkannten Textes sowie das Speichern und Exportieren der Beiträge.

- **Startseite:** Die Startseite der Anwendung dient als zentrale Anlaufstelle und bietet den Nutzern schnellen Zugriff auf die Foto-Funktion, die Galerie sowie die Historie gespeicherter Beiträge (siehe Abbildung 6.2a).

- **Cropping- und Rotier-Funktion:** Die Cropping-Funktion ermöglicht es den Nutzern, den relevanten Bildausschnitt durch einen verschieb- und skalierbaren Auswahlrahmen festzulegen. Dieser kann mittels Touch-Gesten angepasst werden. Ergänzend dazu erlaubt die Rotier-Funktion das Drehen oder Spiegeln des Bildes (siehe Abbildung 6.2b).

- **Pre-OCR-Seite:** Über den Scan-Button kann der OCR-Prozess angestoßen werden (siehe Abbildung 6.2c).

- **OCR-Ergebnis mit Übersetzung und Furigana:** Nach dem Auslösen des Scan-Vorgangs erkennt die Anwendung mithilfe optischer Zeichenerkennung (OCR) die im Bild enthaltenen japanischen Schriftzeichen. Der erkannte Text wird anschließend automatisch ins Englische übersetzt. Die Ergebnisse der Texterkennung und Übersetzung werden in zwei getrennten Textfeldern dargestellt. Über eine zusätzliche Button können erkannte Kanji optional mit Furigana ergänzt werden (siehe Abbildung 6.2d).

- **Editierfunktion:** Es kann vorkommen, dass die OCR-API fehlerhafte Ergebnisse liefert. Damit die Qualität der Übersetzung darunter nicht leidet, kann über einen Button die Editierfunktion aktiviert und der erkannte Text manuell angepasst werden (siehe Abbildung 6.2e).

- **Historie:** Beiträge, die abgespeichert wurden, erscheinen in der Historie. Der Nutzer kann Beiträge auf sein Endgerät als PDF exportieren sowie einzelne oder alle Beiträge löschen (siehe Abbildung 6.2f).



(a) (b) (c)

(d) (e) (f)

Abbildung 6.2: (a) Hauptmenü der App. (b) Auswahl eines Bildausschnitts mittels Cropping-Funktion vor der Texterkennung. (c) Pre-OCR-Seite mit Scan-Button zur Initiierung des OCR-Prozesses. (d) Anzeige des erkannten japanischen Textes mit aktivierter Furigana-Darstellung sowie der automatisch erzeugten englischen Übersetzung. (e) Aktivierte Editierfunktion zur manuellen Korrektur des OCR-Ergebnisses. (f) Übersicht der gespeicherten Beiträge in der Historienansicht. Nutzer können Beiträge hier als PDF exportieren oder löschen.

6.3 Technische Einschränkungen und offene Aufgaben

Die folgende Übersicht fasst zentrale technische Defizite und noch offene Aufgaben zusammen, die für eine produktive Nutzung oder Veröffentlichung der App noch gelöst werden müssten.

- **Plattformbeschränkung:** Die Anwendung wurde ausschließlich für Android entwickelt. Eine Version für iOS konnte nicht umgesetzt werden, da hierfür ein kostenpflichtiger Apple-Developer-Account erforderlich ist, der im Rahmen dieses Projekts nicht zur Verfügung stand.

- **Verlässlichkeit der sprachtechnologischen Komponenten:** Die sprachverarbeitenden Module – OCR, Furigana-Erzeugung und Übersetzung – liefern im Zusammenspiel unzuverlässige Ergebnisse. Während die Furigana-Komponente in den meisten Fällen korrekte Lesungen erzeugt, ist die OCR insbesondere bei handschriftlichen oder vertikalen Texten fehleranfällig. Auch die Übersetzungskomponente weist Schwächen auf, vor allem bei Redewendungen, literarischen oder ungewöhnlich strukturierten Texten. Damit die Anwendung als zuverlässiges Werkzeug eingesetzt werden kann, müssten im Fall von OCR und Übersetzung leistungsstärkere Technologien eingesetzt werden.

- **Datenschutz:** Die Kommunikation mit dem eigenen Übersetzungsserver sowie mit allen weiteren potenziell eingesetzten externen Servern muss im Falle einer Veröffentlichung sorgfältig abgesichert und datenschutzkonform gestaltet werden. Eine entsprechende Prüfung und Umsetzung wurde bislang noch nicht durchgeführt.

- **Sicherheitsrisiken:** Fehlende Input-Validierung kann dazu führen, dass schädlicher Code eingeschleust wird (*Code Injection*) oder andere Angriffe stattfinden. Im Rahmen dieses Projekts wurden solche Risiken bisher nicht analysiert.

- **Keine ICR-Unterstützung:** Die Integration von *Intelligent Character Recognition* (ICR) zur Verbesserung der OCR-Ergebnisse wurde zwar geplant, ist jedoch bisher nicht umgesetzt worden.

- **Fehlende Bildoptimierung:** Funktionen zur Verbesserung der Bildqualität, wie Kontrastanpassung und Schärfung, wurden bisher nicht integriert.

7

Zusammenfassung

Diese Projektarbeit dokumentiert die Konzeption, Entwicklung und Evaluation einer mobilen Anwendung, die japanische Texte in Bildern erkennt, mit Furigana-Lesehilfen versieht und ins Englische übersetzt. Die App richtet sich an Sprachlernende und Reisende, die im Alltag mit japanischen Texten konfrontiert sind. Zur Umsetzung wurden React Native und Expo verwendet, was eine plattformübergreifende Entwicklung für Android und iOS ermöglicht. Im Projekt wurde die App jedoch ausschließlich für Android realisiert.

Vor der Implementierung erfolgte eine systematische Anforderungsanalyse. Für die Texterkennung kam Google ML Kit zum Einsatz, wobei vertikale japanische Texte eine besondere Herausforderung darstellten. Die Furigana wurden mithilfe der Bibliothek Kuromoji und der Transkriptionsbibliothek Wanakana generiert. Die Übersetzung übernahm ein serverseitiges Modell aus der Helsinki-NLP-Reihe von Hugging Face.

Nach der Entwicklung wurde die App hinsichtlich Gebrauchstauglichkeit und technischer Leistungsfähigkeit evaluiert. Testpersonen führten typische Nutzungsszenarien durch und beantworteten anschließend einen Fragebogen zu Bedienbarkeit und Verständlichkeit. Die technische Analyse untersuchte die Genauigkeit der Texterkennung sowie die Qualität der generierten Furigana und Übersetzungen anhand der Metriken CER, BLEU und COMET. Ergänzend erfolgte eine Fehleranalyse zur Identifikation von Schwächen.

Die Anwendung zeigt sich als vielversprechender Ansatz, der mit gezielten technischen Verbesserungen zu einem effektiven Lernbegleiter ausgebaut werden kann.

Literaturverzeichnis

Bee06. Beermann, Rafael E.: *Introduction to the Japanese Writing System: 文字入門*, 2006. Unterrichtsmaterial, Universität Duisburg-Essen (ehemals Gerhard-Mercator-Universität-GH Duisburg).

Han16. Han, Lifeng: *Machine Translation Evaluation Resources and Methods: A Survey*, 2016.

Her22. Herrmann, Andrea: *Grundlagen der Anforderungsanalyse: Standardkonformes Requirements Engineering*. Springer Vieweg, Wiesbaden, 2022.

HMK16. Hussain, Azham, Emmanuel O. C. Mkpojiogu und Fazillah Mohmad Kamal: *The Role of Requirements in the Success or Failure of Software Projects*. International Review of Management and Marketing, 6, 2016.

IEE90. IEEE Standards Board: *IEEE Standard Glossary of Software Engineering Terminology*. IEEE Std 610.12-1990, 1990. DOI: 10.1109/IEEESTD.1990.101064.

JCI14. Jana, Ranjan, Amrita Roy Chowdhury und Mazharul Islam: *Optical Character Recognition from Text Image*. International Journal of Computer Applications Technology and Research, 3:239–243, 2014.

LA24. Limantoro, Gray Hansen und Yeyen Dwi Atma: *History of Japanese Writing System: From Kanji Into Hiragana*. International Journal of Multidisciplinary Sciences and Arts, 3, 2024.

LLM+23. Lee, Seungjun, Jungseob Lee, Hyeonseok Moon, Chanjun Park, Jaehyung Seo, Sugyeong Eo, Seonmin Koo und Heuiseok Lim: *A Survey on Evaluation Metrics for Machine Translation*. Mathematics, 11(4):1006, 2023.

NSCT25. Nagaonkar, Sankalp, Augustya Sharma, Ashish Choithani und Ashutosh Trived: *Benchmarking Vision-Language Models on Optical Character Recognition in Dynamic Video Environments*. 2025.

NZB+21. Neudecker, Clemens, Karolina Zaczynska, Konstantin Baierer, Georg Rehm, Mike Gerber und Julián Moreno Schneider: *Methoden und Metriken zur Messung von OCR-Qualität für die Kuratierung von Daten und Metadaten*. In: *Qualität in der*

Inhaltserschließung, Seiten 137–166. De Gruyter Saur, Berlin/Boston, 2021.

Pax19. Paxton, Simon: *Kanji Matters in a Multilingual Japan.* Bulletin of the Center for Foreign Language Education and Research, Rikkyo University, 2019.

PR21. Pohl, Klaus und Chris Rupp: *Basiswissen Requirements Engineering: Aus- und Weiterbildung nach IREB-Standard zum Certified Professional for Requirements Engineering Foundation Level.* dpunkt.verlag, Heidelberg, 5. Auflage, 2021.

PRWZ01. Papineni, Kishore, Salim Roukos, Todd Ward und Wei-Jing Zhu: *BLEU.* In: Isabelle, Pierre (Herausgeber): *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics - ACL '02*, Seite 311, Morristown, NJ, USA, 2001. Association for Computational Linguistics.

Ros17. Rose, Heath: *The Japanese Writing System: Challenges, Strategies and Self-regulation for Learning Kanji.* Multilingual Matters, Cambridge, Massachusetts, 2017.

RRWD22. Roux, Thibault Baneras, Mickael Rouvier, Jane Wottawa und Richard Dufour: *Qualitative Evaluation of Language Model Rescoring in Automatic Speech Recognition.* 2022.

RSFL20. Rei, Ricardo, Craig Stewart, Ana C. Farinha und Alon Lavie: *COMET: A Neural Framework for MT Evaluation.* In: Webber, Bonnie, Trevor Cohn, Yulan He und Yang Liu (Herausgeber): *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Seiten 2685–2702, Stroudsburg, PA, USA, 2020. Association for Computational Linguistics.

RT22. Rivera-Trigueros, Irene: *Machine translation systems and quality assessment: a systematic review.* Language Resources and Evaluation, 56(2):593–619, 2022.

Sch23. Schrepp, Martin: *User Experience Questionnaire Handbook: All you need to know to apply the UEQ successfully in your projects*, 2023. Version 11 (12.09.2023).

Som11. Sommerville, Ian: *Software Engineering.* Pearson, 9 Auflage, 2011.

VTM23. Vanroy, Bram, Arda Tezcan und Lieve Macken: *MATEO: MAchine Translation Evaluation Online.* In: *Proceedings of the 24th Annual Conference of the European Association for Machine Translation*, Seiten 499–500, Tampere, Finland, Juni 2023. European Association for Machine Translation.

WB13. Wiegers, Karl und Joy Beatty: *Software Requirements.* Microsoft Press, 3 Auflage, 2013.

Zis20. Zisk, Matthew: *A Brief Overview of Japanese Dictionaries, Past and Present.* AJALT, 43, 2020.

A

Eigenständigkeitserklärung

☐ Die vorliegende Arbeit wurde als Einzelarbeit angefertigt.

☒ Die vorliegende Arbeit wurde als Gruppenarbeit angefertigt. Mein Anteil an der Gruppenarbeit ist im untenstehenden Abschnitt *Verantwortliche* dokumentiert:

☒ Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig und ohne unzulässige Hilfe Dritter angefertigt habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche kenntlich gemacht. Darüber hinaus erkläre ich, dass ich die vorliegende Arbeit in dieser oder ähnlicher Form noch nicht als Prüfungsleistung eingereicht habe.

☐ Es ist keine Nutzung von KI-basierten text- oder inhaltgenerierenden Hilfsmitteln erfolgt.

☒ Die Nutzung von KI-basierten text- oder inhaltgenerierenden Hilfsmitteln wurde von der/dem Prüfenden ausdrücklich gestattet. Die von der/dem Prüfenden mit Ausgabe der Arbeit vorgegebenen Anforderungen zur Dokumentation und Kennzeichnung habe ich erhalten und eingehalten. Sofern gefordert, habe ich in der untenstehenden Tabelle *Nutzung von KI-Tools* die verwendeten KI-basierten text- oder inhaltgenerierenden Hilfsmittel aufgeführt und die Stellen in der Arbeit genannt. Die Richtigkeit übernommener KI-Aussagen und Inhalte habe ich nach bestem Wissen und Gewissen überprüft.

Datum

Unterschrift der Kandidatin/des Kandidaten

Verantwortliche

Die Tabellen unten führen auf, wer als Autor für die einzelnen Kapitel der vorliegenden Dokumentation beziehungsweise für einzelne Teile des Quellcodes hauptverantwortlich ist.
Insgesamt beteiligt sind die folgenden Personen:

- Leonie Kaschek
- Nagina Nasir
- Franziska Kaschek

Dokumentation

Kapitel	**Überschrift**	**Autor**
1	Einleitung	Franziska Kaschek
2	Einführung in das japanische Schriftsystem	Franziska Kaschek
3	Anforderungsanalyse	Franziska Kaschek, Nagina Nasir, Leonie Kaschek
4	Technische Konzeption und Implementierung	Franziska Kaschek
4.1	Frameworks	Nagina Nasir
4.2	Verzeichnisstruktur	Franziska Kaschek
4.3.1	Bildimport-Komponenten (Kamera und Galerie)	Franziska Kaschek
4.3.2	Komponente zur Texerkennung (OCR)	Franziska Kaschek
4.3.3	Komponente zur Furigana-Erzeugung	Leonie Kaschek
4.3.4	Übersetzungskomponente	Nagina Nasir
4.3.5	Speicherkomponente	Nagina Nasir
4.3.6	Komponente zum PDF-Export (OCR)	Leonie Kaschek
5	Qualitätssicherung	Franziska Kaschek
5.1	Evaluation der Hauptfunktionen	Franziska Kaschek, Nagina Nasir, Leonie Kaschek
5.2	Usability Testing	Franziska Kaschek
6	Ergebnisse	Franziska Kaschek, Leonie Kaschek

Nutzung von KI-Tools

KI-Tool	**Genutzt für**	**Warum?**	**Wann?**	**Mit welcher Eingabefrage bzw. -aufforderung?**	**An welcher Stelle der Arbeit übernommen?**
ChatGPT	Optimierung eigener Textentwürfe	Rechtschreibkorrektur, verbesserte Lesbarkeit, präzisere Formulierungen	Über die gesamte Arbeit hinweg	Wie kann man diesen Abschnitt optimieren? Bitte formuliere diesen Abschnitt präziser und kürzer.	S. 1 ff.
	Formulierung von Kurzfassung, Zusammenfassung und einigen Kapiteleinleitungen	Effizientes Schreiben	Bei der Bearbeitung der betreffenden Kapitel	Bitte formuliere eine Einleitung zu diesem Kapitel. Bitte formuliere eine Kurzfassung/-Zusammenfassung zu dieser Arbeit.	S. 16, S. 61
	Ausformulierung der Use Cases und Anforderungen (Tabellen) auf Basis von Stichpunkten	Effizientes Schreiben	Bei der Bearbeitung des Anforderungsteils der Arbeit	Bitte formuliere die angegebenen Use Cases und Anforderungen gemäß etablierter Qualitätskriterien.	S. 9-14
	Beschreibung der Verzeichnisstruktur und der Farbpalette	Effizientes Schreiben	Bei der Bearbeitung der betreffenden Kapitel	Bitte beschreibe kurz und präzise die Funktion der angegebenen Ordner und Dateien. Bitte erstelle aus der Farbpalette und den Verwendungszwecken einen kurzen Fließtext.	S. 18-20, S. 57
	Unterstützung bei der Code-Beschreibung	Verständliche Darstellung der Implementierung	Bei der Bearbeitung des Implementierungsteils der Arbeit	Bitte beschreibe die wesentlichen Schritte in diesem Programmabschnitt präzise und verständlich.	S. 21-40